{Learn, Create, Innovate};

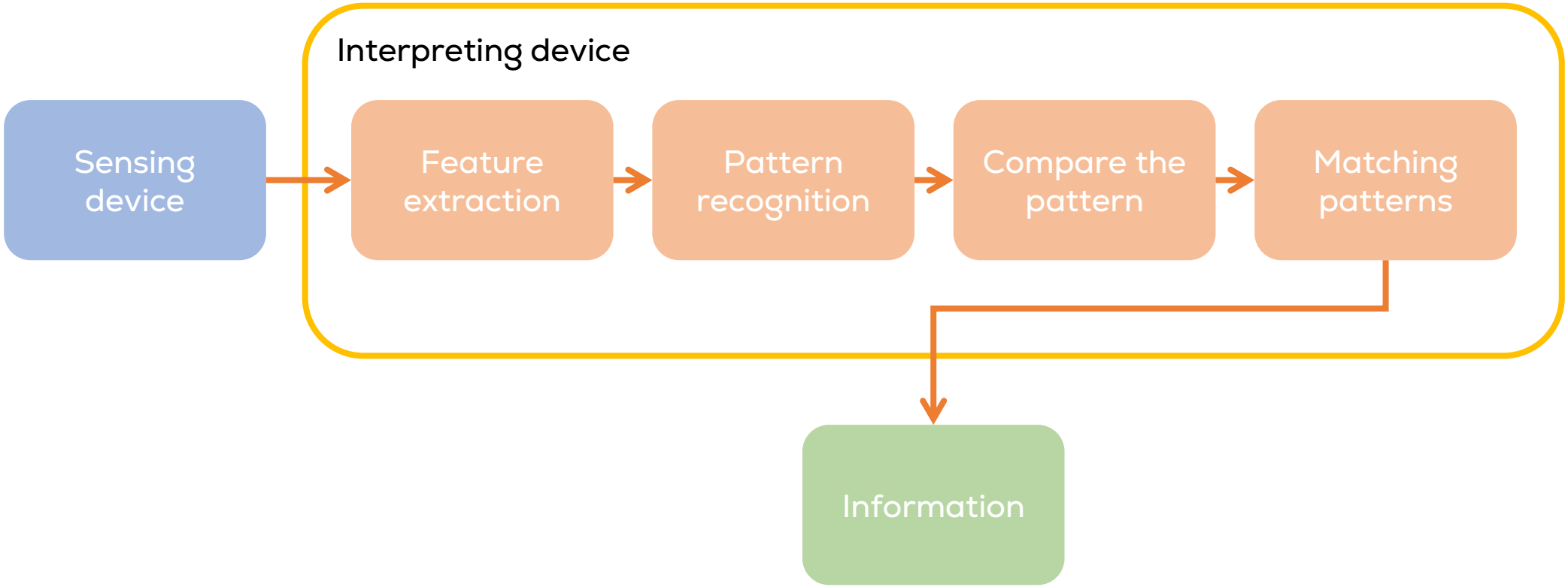
Computer Vision

Introduction

MCR²

Manchester **Robotics**

Computer Vision General Pipeline



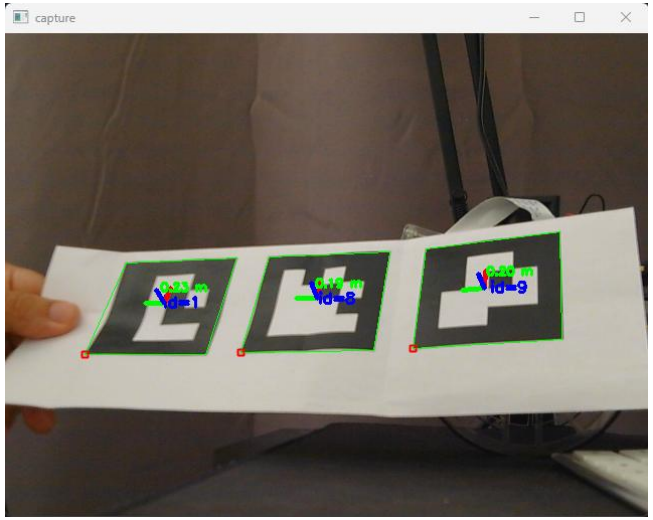
Computer Vision Software

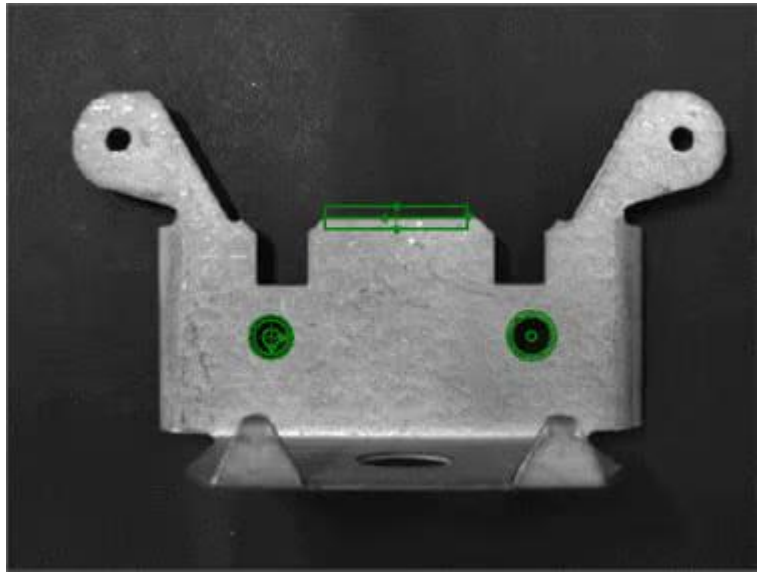
Computer Vision Software

- Modern computer vision applications use a wide range of software tools, each built for different audiences, different workflows, and different levels of abstraction.
- Research & Development: Tools focused on building core CV algorithms, experimenting, and customizing every step.
  - Examples: OpenCV, MATLAB
- Automation & Deployment: Tools designed to solve factory problems quickly and reliably, using pre-built, validated vision tools rather than custom algorithms.
  - Examples: CVL (Cognex Vision Library), HALCON.

Computer Vision Software

- Algorithm-Level CV
  - Developer writes or modifies algorithms
  - High flexibility
  - Requires expertise
  - Ideal for robotics, AI research, custom applications
- Tool-Level CV (Industrial CV)
  - Pre-engineered tools (e.g., OCR, blob analysis, geometric matching, measurement)
  - Fast to configure, highly reliable
  - Limited access to algorithm internals
  - Ideal for manufacturing, inspection, automation





Computer Vision Software

Do we need to learn both?

Yes…

- Research, robotics, and custom applications need algorithm-level control
- Industrial automation and factory deployment need proven, reliable tools
- Understanding both allows engineers to choose the right tool for each challenge.
- Development tools teach how computer vision truly works (filters, thresholds, feature extraction, deep learning).
- Industrial tools teach practical deployment constraints (lighting, robustness, cycle time, reliability).
- Together, they form a complete engineering skill set.

Computer Vision Software

Ability to Diagnose and Improve Industrial Solutions

- Industrial tools hide algorithm internals, so engineers must understand the underlying principles to:
  - Tune parameters correctly
  - Identify failure modes
  - Recognize when a tool is not enough
  - Know when a custom algorithm is needed

Innovation happens at the intersection

- Many modern systems mix both worlds:
- Factory tools + deep learning (OpenCV/TensorFlow/PyTorch)
- Custom algorithms deployed as industrial tools
- Hybrid robotic systems where core CV is custom but final validation is industrial

Computer Vision Software

	OpenCV	Cognex CVL (Industrial)	MATLAB (CV Toolbox)	HALCON
Type	Open-source algorithm library	Industrial toolset with hight level vision tools	Prototyping & algorithm library	Industrial toolset with hight level vision tools
Focus	Core algorithms and low/high-level CV building blocks	Industrial-grade tools, ready for deployment	Algorithm development, prototyping, education	Industrial-grade tools, ready for deployment
Approach	Developers combine primitives/advanced to create solutions	Users configure high-reliability tools (blob, OCR, alignment, etc.)	Developers combine primitives/advanced to create solutions	Users configure high-reliability tools (blob, OCR, alignment, etc.)
Industrial Robustness	Depends on user implementation	Very high	Depends on user implementation	Very high
Speed & Performance	High (optimized C++)	Very high, hardware optimized	Medium	Very high, hardware optimized
Typical Use Cases	Robotics, research, custom CV	Inspection, alignment, barcode/OCR, metrology	Teaching, algorithm research	Inspection, alignment, barcode/OCR, metrology

Computer Vision

OpenCV

{Learn, Create, Innovate};

MCR²

Manchester Robotics

OpenCV

- Open-Source Computer Vision Library
- Cross-platform, free to use
- Originally developed by Intel
- Aimed at real-time computer vision
- Contains a wealth of functions and routines for the real-time processing and analysis of images
- Compatible with ROS

Computer Vision

OpenCV: Colour Spaces

{Learn, Create, Innovate};

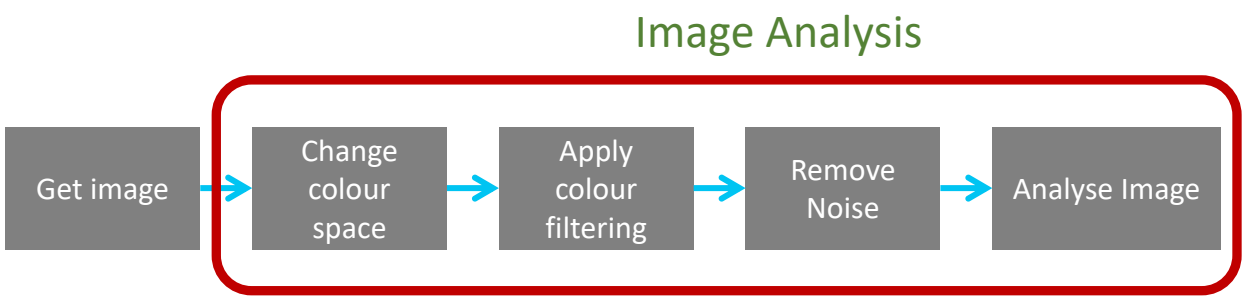
MCR²

Manchester Robotics

Colour Spaces

Colour Spaces

- A **colour space** is a mathematical representation of colours that allows computers to process and analyse images in different ways.



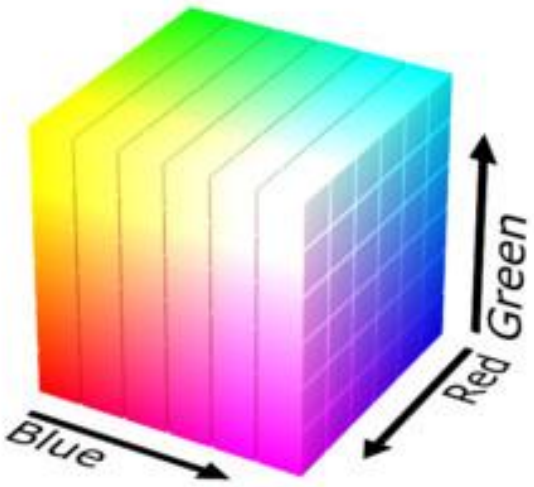
Simple image analysis pipeline that can be implemented in OpenCV.

- OpenCV manages different colour representations (more than 150), called colour spaces.
- The most widely used are BGR (Blue, Green, Red), HSV (Hue, Saturation, Value) and Gray (Grayscale, LAB (CIE L*a*b*)
- OpenCV offers conversions amongst these spaces i.e., BGR ↔ Gray and BGR ↔ HSV, etc.
- More information here.

Colour Spaces

BGR (Blue, Green, Red):

- Default colour format in OpenCV.
- Channels: Blue, Green, Red
- Default format when reading images (cv.imread)
- It is not ideal for colour detection due to the mixing of colour intensities.
- Not robust to illumination changes
- Good for display



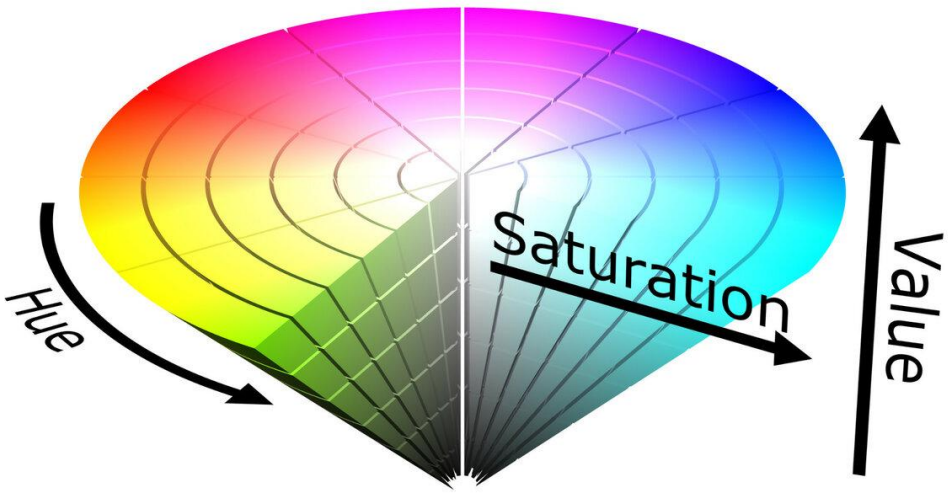

```
img_bgr = cv.imread("image.jpg")   # BGR format
```

Colour Spaces

HSV (Hue, Saturation, Value):

- **Hue (H):** Represents colour type (0 to 179 degrees in OpenCV in other software this may vary).
- **Saturation (S):** Concentration/diluting of colour. Unsaturated to represent shades of grey and fully saturated (no white component). (0 to 255).
- **Value (V):** Brightness or intensity of colour (0 to 255).

- Why HSV?
  - More intuitive way to define/describe colours.
  - Separates colour from brightness
  - Very robust for colour detection & segmentation
  - Widely used in robotics and industrial vision
  - Used to perform colour-based segmentation




```
hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)
```

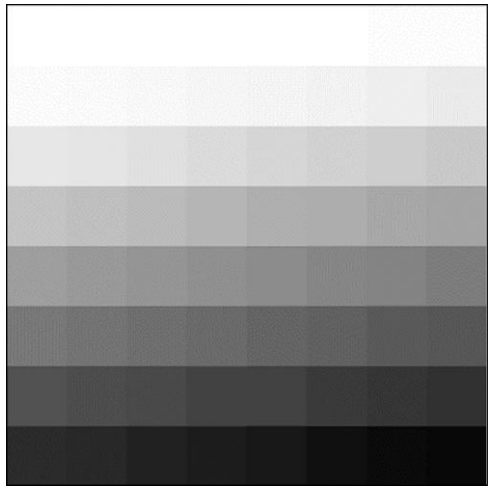
Colour Spaces

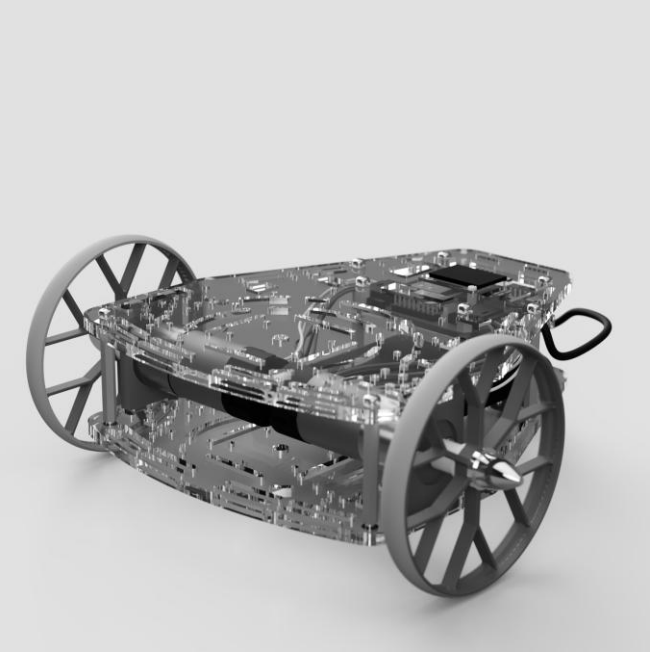
Grayscale:

- Gray scaling is the process of converting an image from other colour spaces e.g. RGB, CMYK, HSV, etc. to shades of grey.
- It varies between complete black and complete white.
- Single intensity channel
- Standard OpenCV:

$$RGB[A] to\ Gray: Y \leftarrow 0.299R + 0.587\ G + 0.114B$$

- Why Grayscale?
  - Reduces complexity.
  - Some methods use grayscale.





```
img_gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)
```

Activity 1

Colour Spaces

{Learn, Create, Innovate};

MCR²

Manchester Robotics

Colour Spaces

- The objective of this activity is to compare the different colour spaces.
- Make a Python file called “opencv_imgColors.py”
- Open your preferred IDE or a notepad.
- Save your file as “.py”
- Make sure the previous images are in the folder “opencv_course/images”
- Copy the code in the next slide
- Launch it from terminal in Windows or Ubuntu respectively:

```
$ python opencv_imgOverlay.py
```

```
$ python3 opencv_imgOverlay.py
```

```python
import cv2 as cv
import numpy as np

# Load image (OpenCV loads in BGR)
img = cv.imread("images/MCR2_Logo_Black.png")

if img is None:
    print("Failed to load image")
    exit()

# Convert to HSV
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)
# Convert to Grayscale
gray = cv.cvtColor(img, cv.COLOR_BGR2GRAY)
# Convert BGR to LAB
lab = cv.cvtColor(img, cv.COLOR_BGR2Lab)

# Display each version in its own window
cv.imshow("Original (BGR)", img)
cv.imshow("HSV", hsv)
cv.imshow("Grayscale", gray)
cv.imshow("LAB", lab)

cv.waitKey(0)
cv.destroyAllWindows()
```

Colour Spaces

Computer Vision

OpenCV: Basic Operations

{Learn, Create, Innovate};

MCR²

Manchester Robotics

Thresholding, Masks and Bitwise Operations

Bitwise Operations

- Bitwise operations work by comparing the **binary representation** of pixel values bit by bit.

- In OpenCV, these operations are typically used to **combine two images** or **apply masks** to selectively show or hide parts of an image.

Thresholding, Masks and Bitwise Operations

Bitwise Operations: AND

- Takes two images (src1 and src2) and performs an AND operation between corresponding pixels.
- If both pixel values are non-zero (i.e., 1), the result is non-zero (i.e., 1). Otherwise, it's zero.
- AND Example:

```
Pixel A --> 11101010 (binary)
Pixel B --> 01010101 (binary)
-----------------------------
Result --> 01000000
```

- In OpenCV, these operations are typically used to apply a mask to preserve specific areas of an image.
- Extract parts of an image that satisfy certain conditions.
- In colour images AND is performed in each channel separately.

```
result = cv2.bitwise_and(image, image, mask=mask)

## If mask used Only pixels where mask is white (255) will be visible in result.The black parts of the mask (0) hide pixels in the output..
```

Thresholding, Masks and Bitwise Operations

```
result = cv2.bitwise_or(img1, img2)
```

Bitwise Operations: OR

- Performs an OR operation between two images.
- If either pixel is non-zero, the result is non-zero.
- OR Example:

  Pixel A --> 0 0 1 0 1 0 1 0 (binary)
  Pixel B --> 0 1 0 1 0 0 0 0 (binary)

  Result --> 0 1 1 1 1 0 1 0

- In OpenCV, these operations are typically used to Combine two images or regions.

Thresholding, Masks and Bitwise Operations

```
result = cv2.bitwise_xor(img1, img2)
```

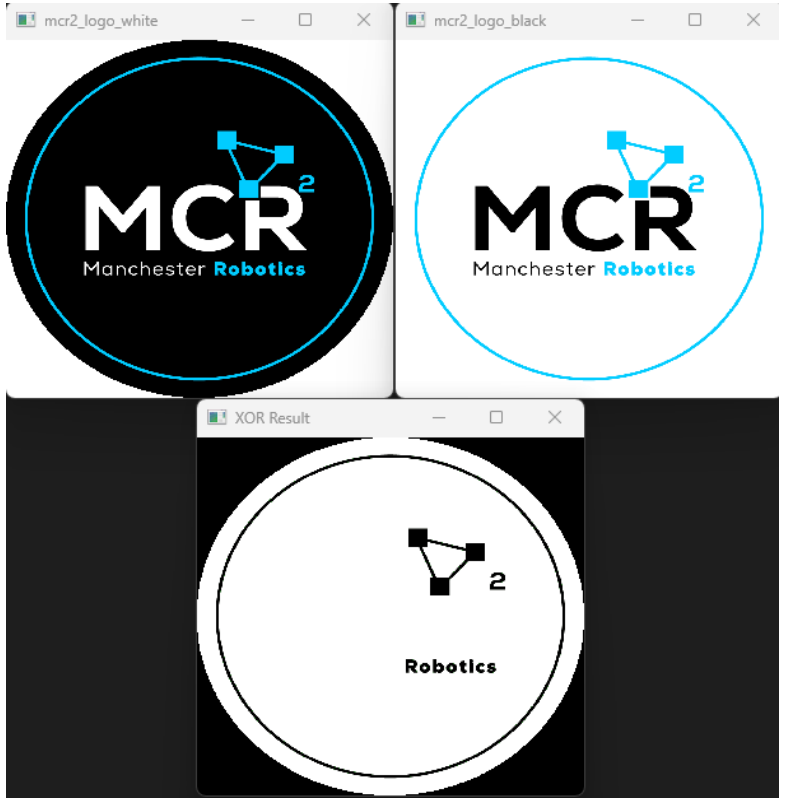
Bitwise Operations: XOR

- Performs an XOR operation. If only one of the pixels is non-zero, the result is non-zero.
- If both are zero or both are non-zero, the result is zero.
- XOR Example:

  Pixel A --> 0 0 1 0 1 0 1 0 (binary)
  Pixel B --> 0 1 0 1 0 0 1 0 (binary)

  Result --> 0 1 1 1 1 0 0 0

- In OpenCV, these operations are typically used to Highlight differences between two images or regions.



Thresholding, Masks and Bitwise Operations

Bitwise Operations: NOT

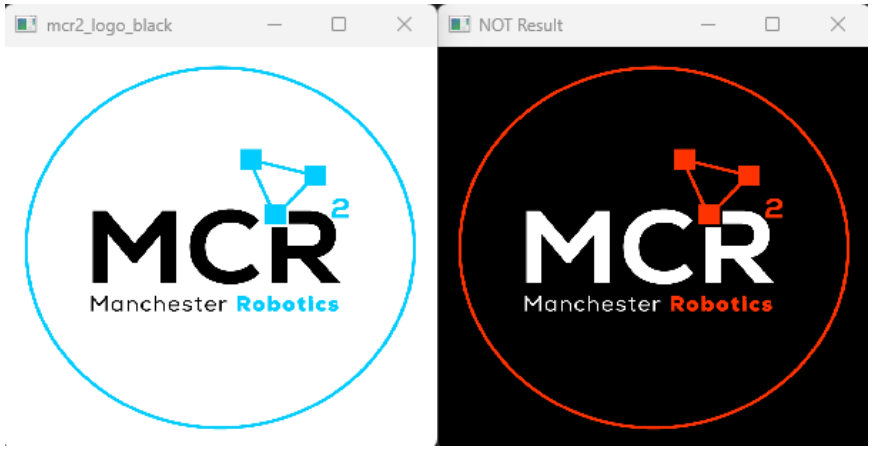
- Inverts every bit of the image.
- Converts white pixels (255) to black (0) and vice versa.
- NOT Example:

  Pixel A --> 0 0 1 0 1 0 1 0 (binary)

  Result --> 1 1 0 1 0 1 0 1

- In OpenCV, these operations are typically used Invert a mask or an image.

```
result = cv2.bitwise_not(img1)
```



Thresholding, Masks and Bitwise Operations

Add

- You can add two images with the OpenCV function, cv.add(), where the res = img1 + img2.
- Both images should be of same depth and type, or the second image can be a scalar value.

```
dst = cv.add(img1_bg,img2_fg).
```

Thresholding, Masks and Bitwise Operations

Thresholding

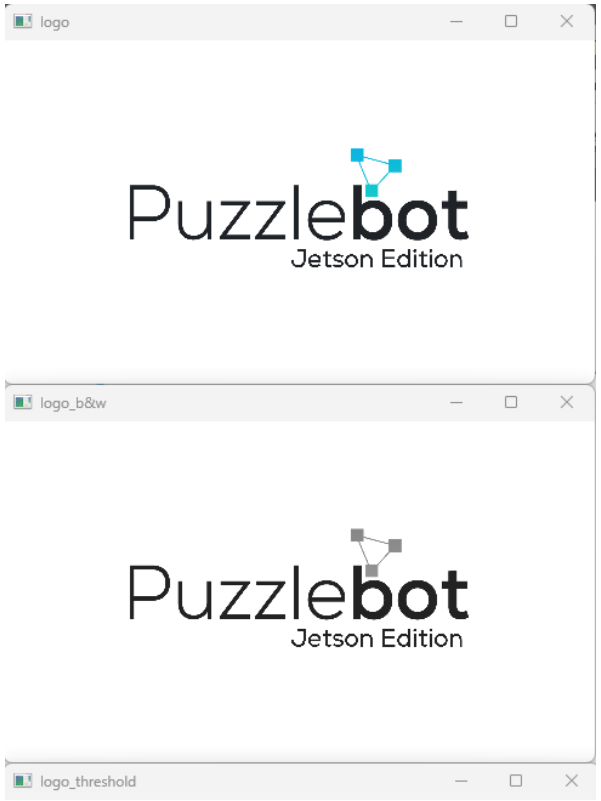
Converts a grayscale image into a binary image. More information here.

Basic Idea:

- If pixel intensity > threshold, set to max value (e.g., 255).
- Else, set to 0.

- Function: `cv2.threshold(source, threshold (0-255), max_value, threshold_type)`

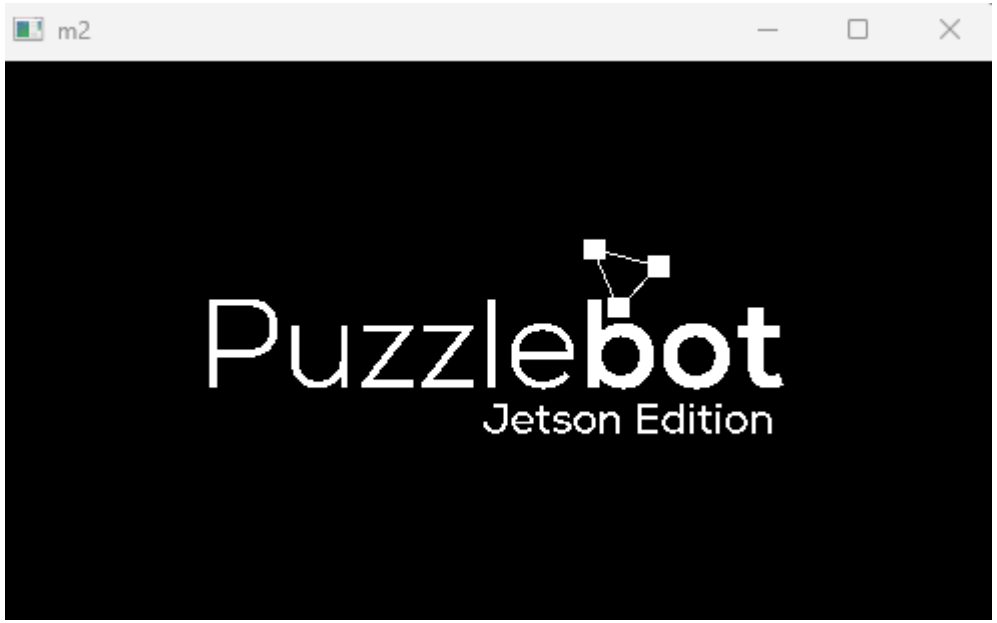
```
img_gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)

ret, mask = cv2.threshold(img_gray, 10, 255, cv2.THRESH_BINARY)

## ret: the threshold used.
## mask: binary image result.
```



Thresholding, Masks and Bitwise Operations

Masks

- A binary image (black & white) used to select parts of another image.
- Usually:
  - White (255): Areas to keep/process.
  - Black (0): Areas to ignore/mask out.
- Example Usage:
  - Focus operations on areas of interest.
  - Create effects and isolate objects.



Activity 2

Simple Masking Exercise

{Learn, Create, Innovate};

MCR²

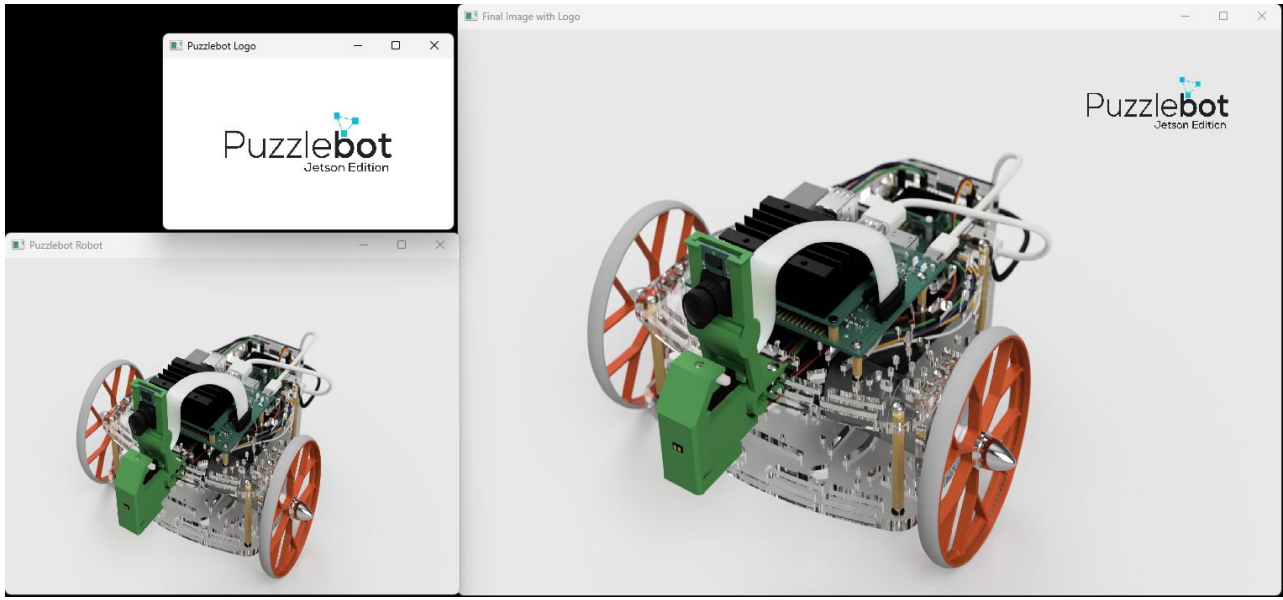
Manchester **Robotics**

Simple Masking Exercise

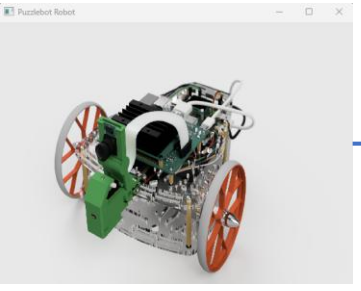
- The objective of this activity is to use masks to overlay images in certain regions of interest.
- Make a Python file called “opencv_imgOverlay.py”
- Open your preferred IDE or a notepad.
- Save your file as “.py”
- Make sure the previous images are in the folder “opencv_course/images”
- Copy the code in the next slide and launch it from terminal in Windows or Ubuntu respectively:

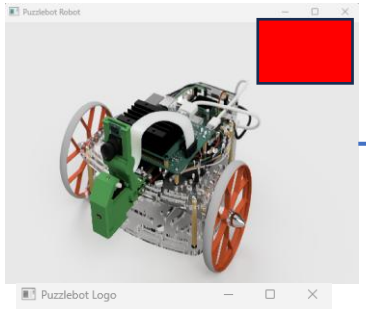
```
$ python opencv_imgOverlay.py
```

```
$ python3 opencv_imgOverlay.py
```



Simple Masking Exercise





Start

Load images

Resize Logo Image (600x300)

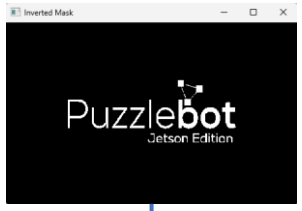
Images Loaded?

Images not loaded

Define ROI on base image (Top Right Corner)

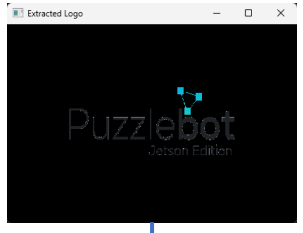
Convert logo to grayscale

Threshold grayscale logo -> Binary mask



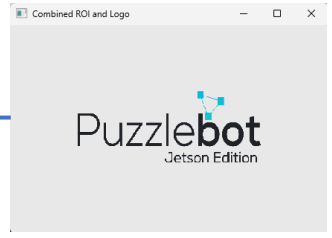
Invert mask

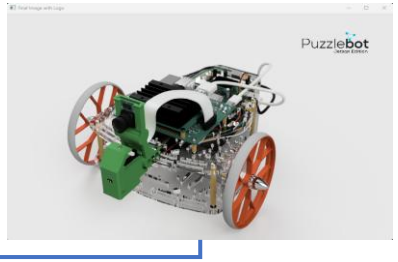
Clear ROI background using mask (AND)



Extract logo foreground using inverse mask (AND)

Add background + logo -> combined ROI





Replace ROI in base image

Display final image

End

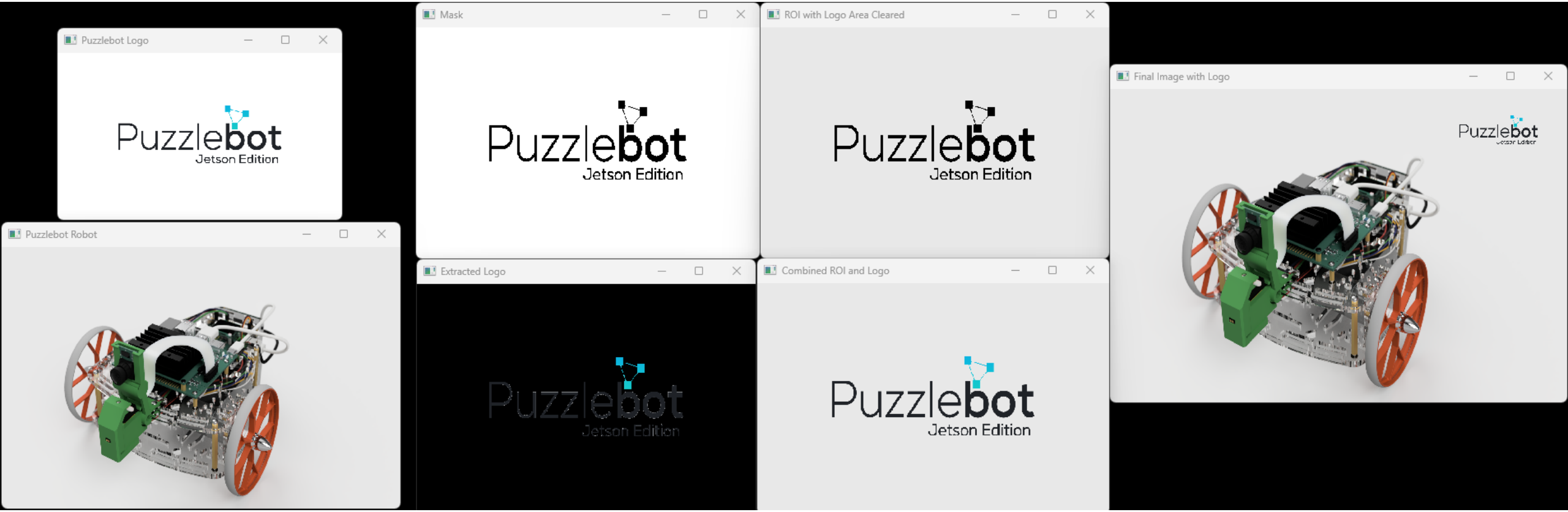
Simple Masking Exercise

```python
import numpy as np
import cv2 as cv

# Load Images
# Base image: Puzzlebot robot render
img1 = cv.imread('images/MCR2__Puzzlebot_Jetson_Ed.png')

# Logo image to overlay on img1
img2 = cv.imread('images/Puzzlebot_logo.png')

# Resize img2 (logo) to desired dimensions (width=600px, height=300px)
img2 = cv.resize(img2, (600, 300))

# Validate if images are loaded correctly
assert img1 is not None, "Error: Base image (img1) could not be read."
assert img2 is not None, "Error: Logo image (img2) could not be read.“

# Show Original Images
cv.namedWindow('Puzzlebot Robot', cv.WINDOW_NORMAL)
cv.imshow('Puzzlebot Robot', img1)

cv.namedWindow('Puzzlebot Logo', cv.WINDOW_NORMAL)
cv.imshow('Puzzlebot Logo', img2)
```

```python
# Create ROI (Region of Interest) in the top-right corner of img1
rows1, cols1, channels1 = img1.shape   # Base image dimensions
rows, cols, channels = img2.shape      # Logo image dimensions

# Select ROI area from img1 where the logo will be placed
roi = img1[0:rows, cols1 - cols:cols1]

# Create Mask and Inverse Mask from the Logo
# Convert logo image to grayscale (for thresholding)
img2_gray = cv.cvtColor(img2, cv.COLOR_BGR2GRAY)

# Threshold to create the mask (white logo area)
ret, mask = cv.threshold(img2_gray, 190, 255, cv.THRESH_BINARY)

# Invert mask (black logo area)
mask_inv = cv.bitwise_not(mask)

# Show mask
cv.namedWindow('Mask', cv.WINDOW_NORMAL)
cv.imshow('Mask', mask)

# Black-out logo area on ROI and Extract the Logo from img2
# Background of ROI with logo area blacked out (using mask)
img1_bg = cv.bitwise_and(roi, roi, mask=mask)

cv.namedWindow('ROI with Logo Area Cleared', cv.WINDOW_NORMAL)
cv.imshow('ROI with Logo Area Cleared', img1_bg)
```

Simple Masking Exercise

```
# Extract the logo from img2 (using inverse mask)
img2_fg = cv.bitwise_and(img2, img2, mask=mask_inv)

cv.namedWindow('Extracted Logo', cv.WINDOW_NORMAL)
cv.imshow('Extracted Logo', img2_fg)

# Combine Cleared ROI and Extracted Logo, then Update Base Image

# Add the cleared ROI and logo together to form final overlay
dst = cv.add(img1_bg, img2_fg)

cv.namedWindow('Combined ROI and Logo', cv.WINDOW_NORMAL)
cv.imshow('Combined ROI and Logo', dst)

# Replace the ROI in img1 with the new combined image
img1[0:rows, cols1 - cols:cols1] = dst

# Show Final Result
cv.namedWindow('Final Image with Logo', cv.WINDOW_NORMAL)
cv.imshow('Final Image with Logo', img1)

# Wait for keypress and close all windows
cv.waitKey(0)
cv.destroyAllWindows()
```


Puzzlebot Logo
Puzzlebot
Jetson Edition
Mask
Puzzlebot
Jetson Edition
ROI with Logo Area Cleared
Puzzlebot
Jetson Edition
Final Image with Logo
Puzzlebot
Puzzlebot Robot
Extracted Logo
Puzzlebot
Jetson Edition
Combined ROI and Logo
Puzzlebot
Jetson Edition

OpenCV

Colour Detection

{Learn, Create, Innovate};

MCR²

Manchester Robotics

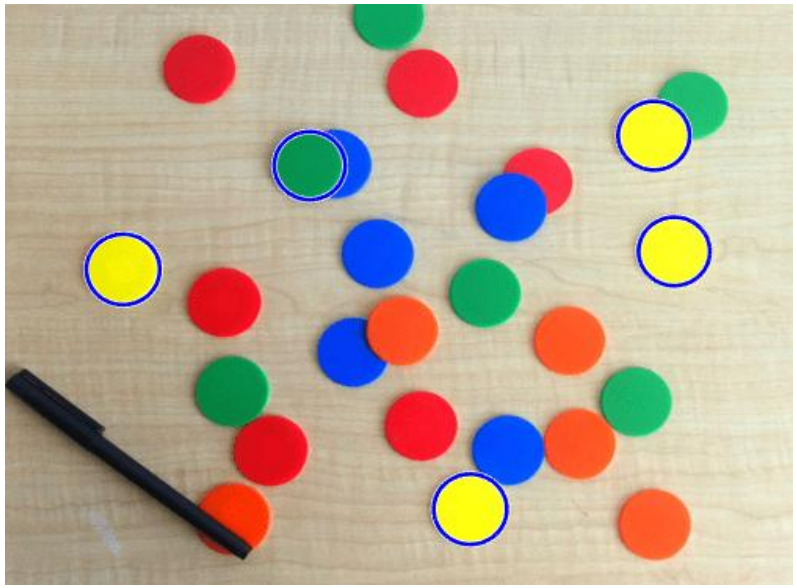
Colour Detection: What is the problem?

Colour Detection

Colour is one of the **simplest and fastest features** to detect:

- Object tracking
- Sorting & inspection
- Robotics perception
- AR overlays

- We aim to detect connected regions in the space with a characteristic colour palette.

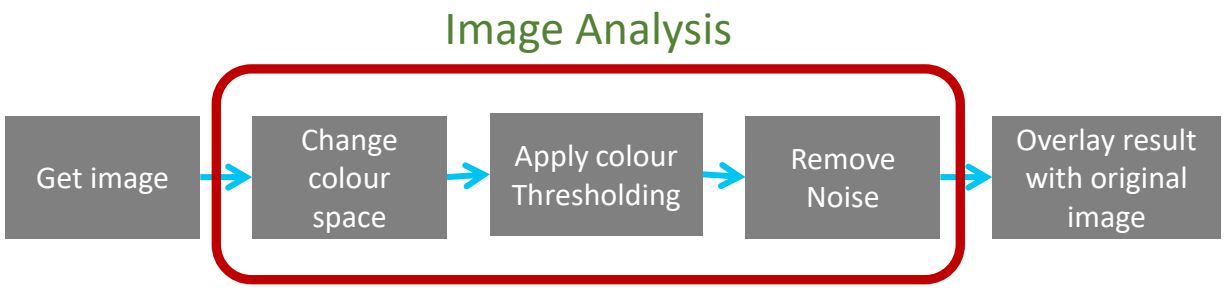


Example of the problem

Colour Detection

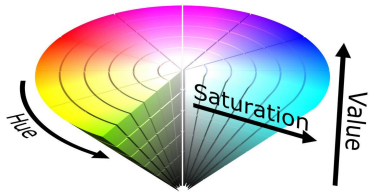
Colour Detection Pipeline

- The following image, shows a simple colour detection pipeline that can be implemented in OpenCV.



Colour Space HSV

- As stated previously one of the most used colour space is the HSV space (more exist!).




```
hsv_image = cv.cvtColor(image,
cv.COLOR_BGR2HSV)
```

- Hue (H): Colour Type, Saturation(S): Colour Purity and Value (V): Brightness tolerance
  - Separates colour from brightness
  - Very robust for colour detection & segmentation
  - Widely used in robotics and industrial vision
  - Used to perform colour-based segmentation

Colour Detection

Colour Range Thresholding

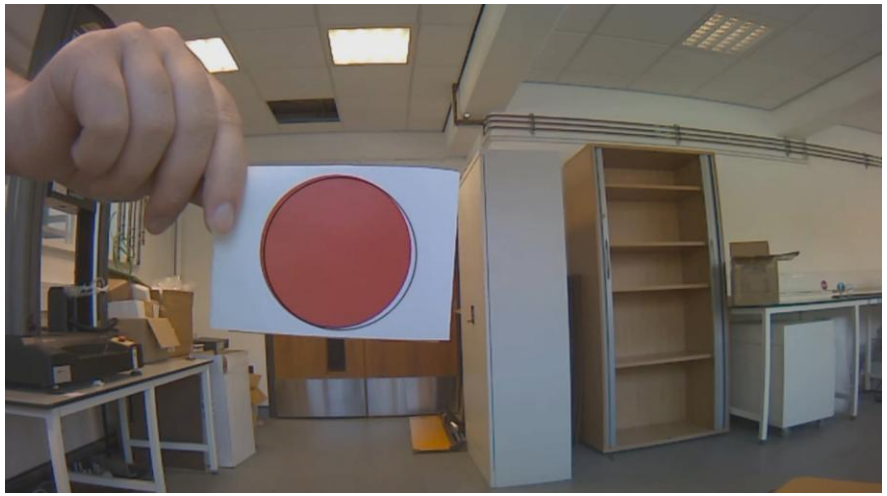
- Now that the colour is on a more practical scale, colours can be filtered by range using the “inRange” function.

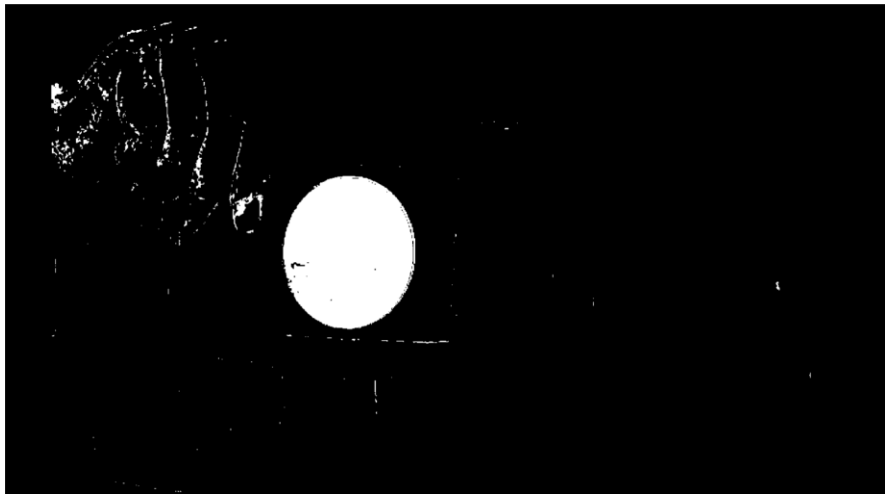
Basic Idea:

- Check if a pixel $[H, S, V]$ values are within a range i.e.,

$$mask(x,y)\begin{cases}255, & if \quad [H_l, S_l, V_l] \leq [H, S, V] \leq [H_u, S_u, V_u] \\ 0, & otherwise\end{cases}$$

```
#Blue Ranges
blue_lower_limit = np.array([90 90, 90])
blue_upper_limit = np.array([110,255,255])

#Blue Mask
blue_mask = cv.inRange(hsv_image, blue_lower_limit, blue_upper_limit)
```





Activity 3

Colour Filtering

{Learn, Create, Innovate};

MCR²

Manchester Robotics

Colour Filtering

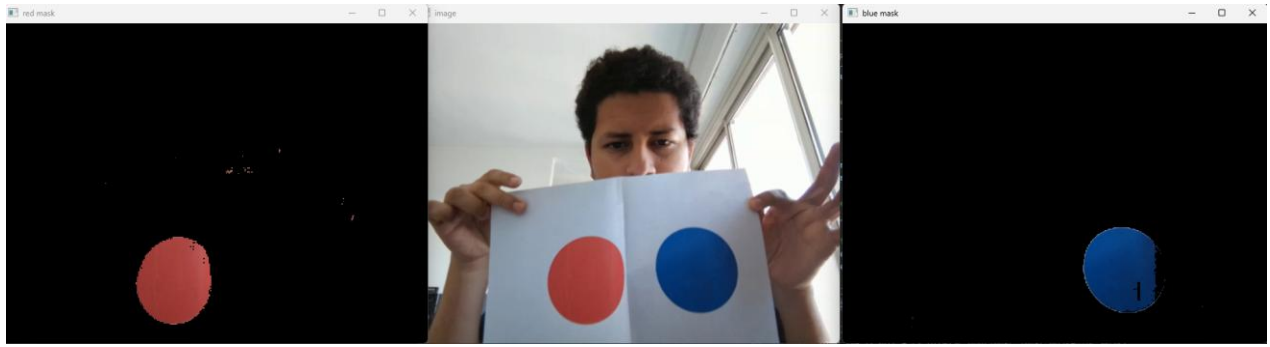
Colour Filtering Algorithm

The Following algorithm will Filter the blue and red colours form the video “sample_video.mp4” (you can use your own).

1. Convert the base image from RGB to the HSV space
2. Filter the image to remove all colours not defined as red or blue, storing the results in two separate images.
    1. Use OpenCV’s `inRange` function
    2. Suggested values for the colours required

	Red	Blue
H	0-33	95-110
S	90-255	80-255
V	130-255	80-255

3. Apply a mask with a bitwise AND operation to view only the colour blue or red.



Colour Filtering

```python
import cv2 as cv
import numpy as np

#HSV Values for Red Colour
H_red_low_limit = 0
S_red_low_limit = 95
V_red_low_limit = 130

H_red_up_limit = 5
S_red_up_limit = 255
V_red_up_limit = 255

#HSV Values for Blue Colour
H_blue_low_limit = 95
S_blue_low_limit = 90
V_blue_low_limit = 90

H_blue_up_limit = 110
S_blue_up_limit = 255
V_blue_up_limit = 255

if __name__=="__main__":
    #cap = cv.VideoCapture(2)
    video_path = 'videos/sample_video.mp4'
    cap = cv.VideoCapture(video_path)

    if not cap.isOpened():
        print("Cannot open camera")
```

```python
    #HSV Values for Green Colour
    while True:
        # Reading frame from video or webcam
        ret, image = cap.read()
        # if frame is read correctly ret is True
        if not ret:
            print("Can't receive frame (stream end?). Exiting ...")
            break
        k = cv.waitKey(25) & 0xFF
        if k == 27:
            break

        # Convert BGR to HSV
        hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)

        #Ranges
        red_lower_limit = np.array([H_red_low_limit, S_red_low_limit, V_red_low_limit])
        red_upper_limit = np.array([H_red_up_limit, S_red_up_limit, V_red_up_limit])
        blue_lower_limit = np.array([H_blue_low_limit, S_blue_low_limit, V_blue_low_limit])
        blue_upper_limit = np.array([H_blue_up_limit, S_blue_up_limit, V_blue_up_limit])

        #Masks
        red_mask = cv.inRange(hsv_image, red_lower_limit, red_upper_limit)
        blue_mask = cv.inRange(hsv_image, blue_lower_limit, blue_upper_limit)

        #Partial masks applied to the image (AND Function with the image)
        red_img_result = cv.bitwise_and(image,image,mask=red_mask)
        blue_img_result = cv.bitwise_and(image,image,mask=blue_mask)

        #Full red mask applied to te image
        cv.imshow('image', image)
        cv.imshow('red mask', red_img_result)
        cv.imshow('blue mask', blue_img_result)

    cap.release()
    cv.destroyAllWindows()
```

OpenCV

Noise Rejection

{Learn, Create, Innovate};

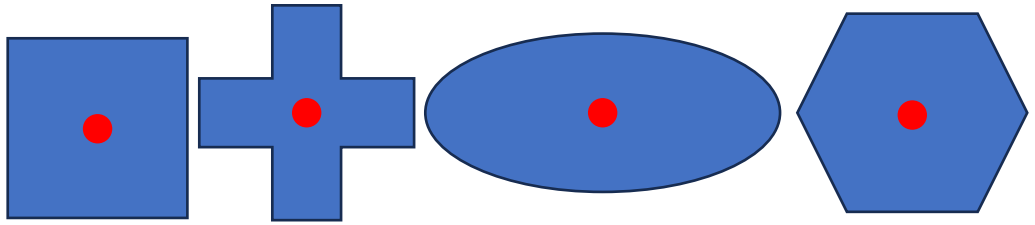
MCR²

Manchester Robotics

Structuring Elements

Kernels (Structuring Elements)

- A Kernel (Structuring element) is shape mask used in different operations.
- Any shape can be a kernel, as long as is digitally representable and has a defined origin.



- It slides over the image
- Defines **which neighbours' matter**
- Controls **shape, size, and direction** of the operation
- Used in:
  - Erosion
  - Dilation
  - Opening
  - Closing
  - Noise removal
  - Shape refinement

Structuring Elements

Kernels (Structuring Elements)

- In image processing, the kernel is a small matrix the defines how a neighbourhood of pixels are processed.
- The kernel can be used to select or convolve

```
# Kernel size
ksize = (11, 11)

# Built-in structuring elements
square  = cv.getStructuringElement(cv.MORPH_RECT, ksize)
cross   = cv.getStructuringElement(cv.MORPH_CROSS, ksize)
ellipse = cv.getStructuringElement(cv.MORPH_ELLIPSE, ksize)
```

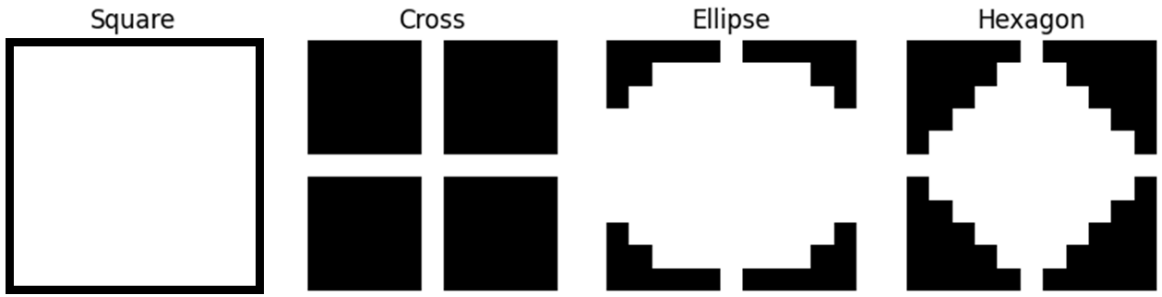
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1
1	1	1	1	1

0	0	1	0	0
0	0	1	0	0
1	1	1	1	1
0	0	1	0	0
0	0	1	0	0

0	0	1	0	0
0	1	1	1	0
1	1	1	1	1
0	1	1	1	0
0	0	1	0	0

0	0	0	0	0
0	1	1	1	0
1	1	1	1	1
0	1	1	1	0
0	0	0	0	0

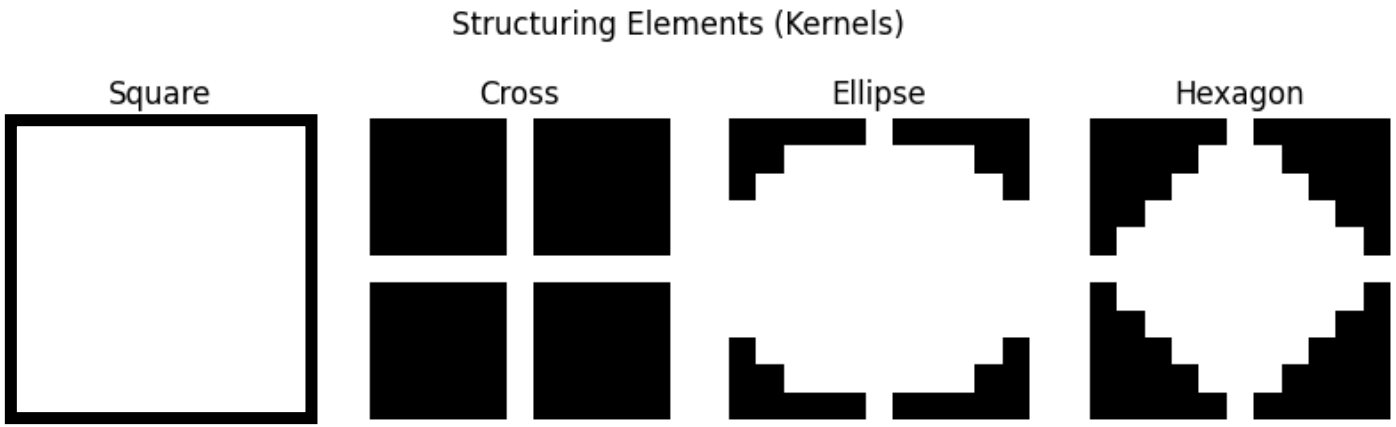
Structuring Elements (Kernels)



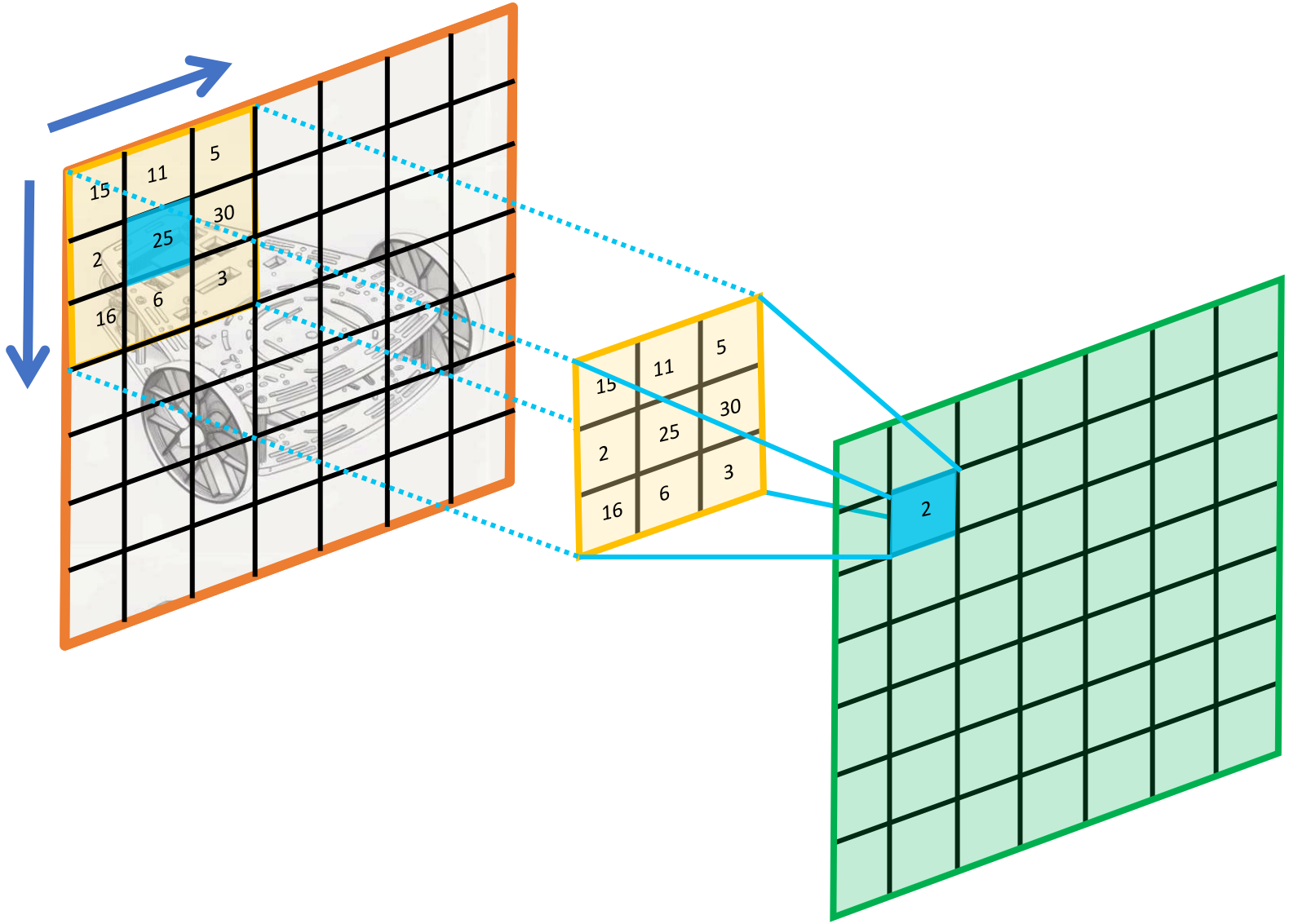
- **The numbers inside the matrix can vary according to the task.**
- **** Other kernel types are possible**

Structuring Elements

- Square Kernel
  - Uniform influence in all directions
  - Aggressive erosion/dilation
  - Best for blocky / digital shapes
- Cross Kernel
  - Only horizontal & vertical neighbours
  - Preserves thin structures
  - Useful for skeletons & grids
- Ellipse Kernel
  - Smooth, rounded influence
  - Preserves natural contours
  - Most common choice in practice
- Hexagon Kernel
  - Isotropic (no strong axis bias)
  - Approximates circular growth
  - Common in advanced morphology & research



Structuring Elements



Noise Rejection

- It is common for noise, such as other objects with the same colour intensity, to appear in the images after applying certain operations.

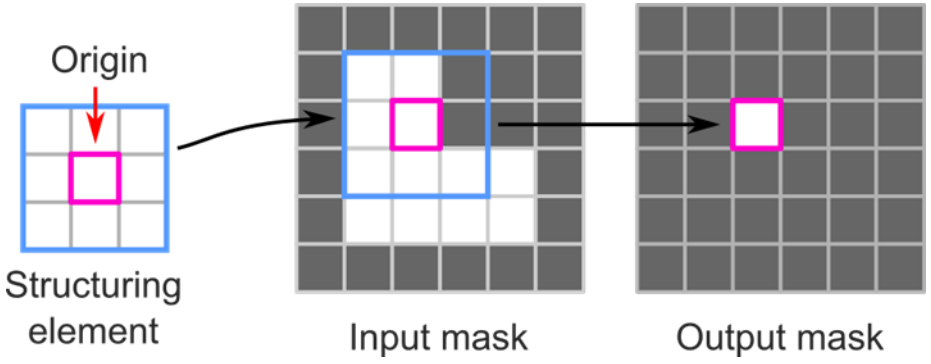
- One of the most common techniques to deal with these artefacts is known as morphological operators.

Noisy image

After some initial operations to an original image, we can see some noise in the result that we want to remove

Morphological Operators

- Morphological operators are defined as a combination between an image and a structuring element.
- The structuring element is usually called “kernel”.
- Some operations can be induced by convolving (convolution) the kernel with the original image.
- The most basic morphological operations are Erosion and Dilation.



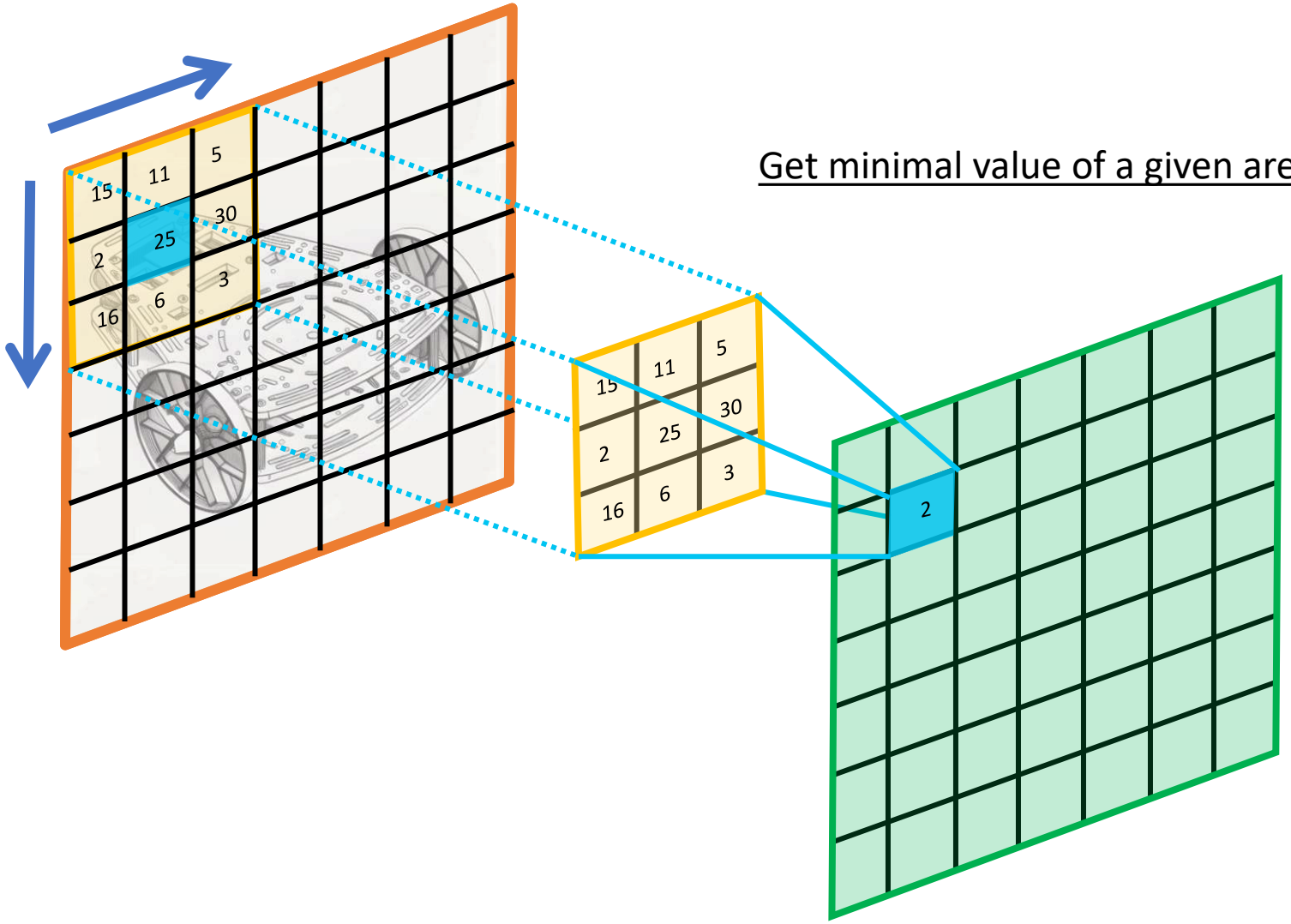
Morphological Operators

Erosion

- It computes a local minimum over the area of a given kernel.
- Using a mask (kernel) , we compute the minimal value of a given area around a pixel, and we replace it with that value.
- The results of this technique are shrinking the larger regions and removing the smallest ones.

```
# Erosion
kernel = cv.getStructuringElement(cv.MORPH_RECT, (5,5))
eroded = cv.erode(mask, kernel, iterations=1)
```

Morphological Operators



Get minimal value of a given area around a pixel

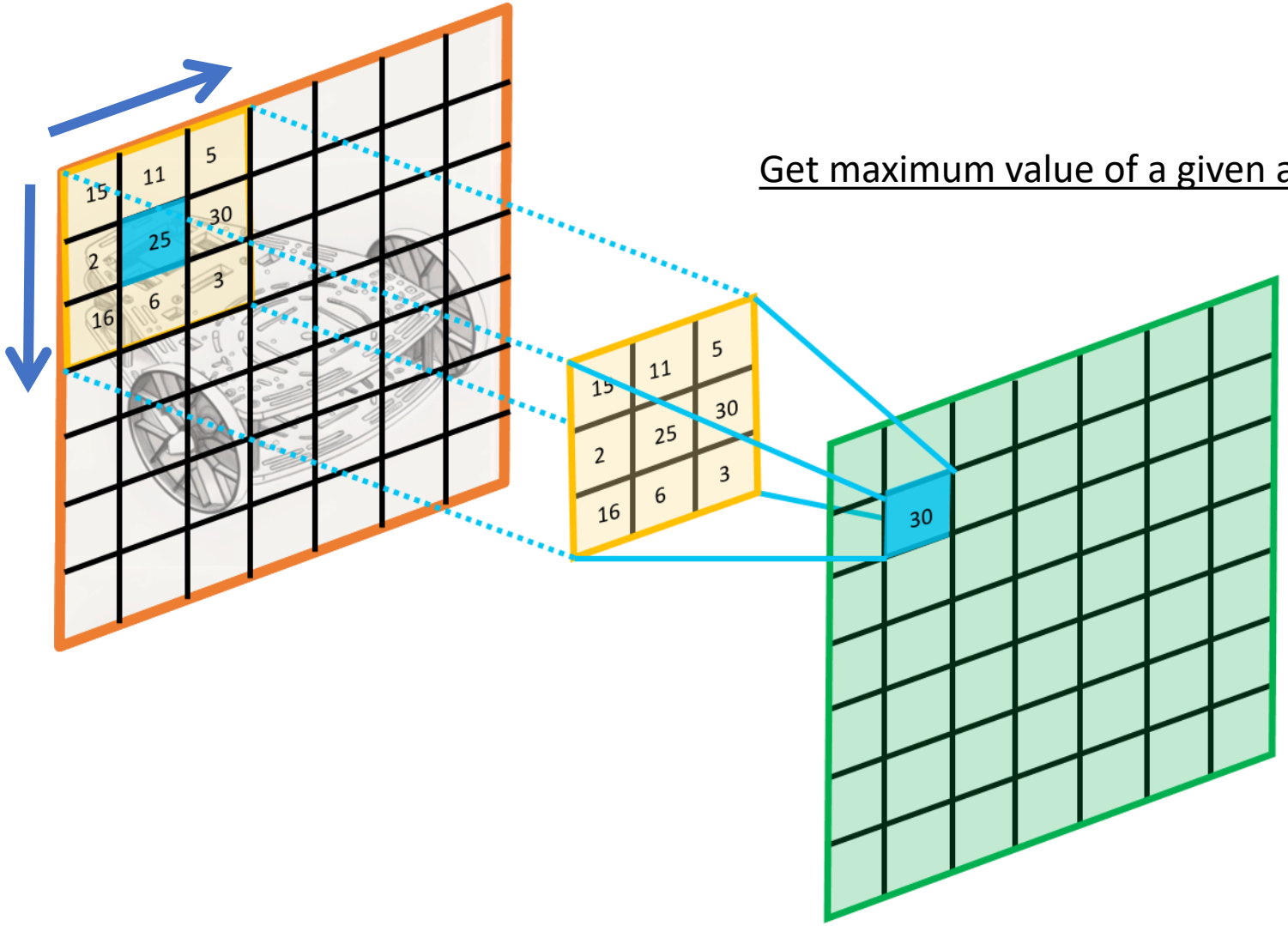
Morphological Operators

Dilation

- Analogous to the Erosion.
- It computes a local maximum over the area of a given kernel.
- Using a mask (kernel), we compute the maximum value of a given area around a pixel, and we replace it with that value.
- The results of this technique are in shapes becoming larger and smoothing their edges.

```
# Dilation
kernel = cv.getStructuringElement(cv.MORPH_RECT, (5,5))
dilated = cv.dilate(mask, kernel, iterations=1)
```

Morphological Operators



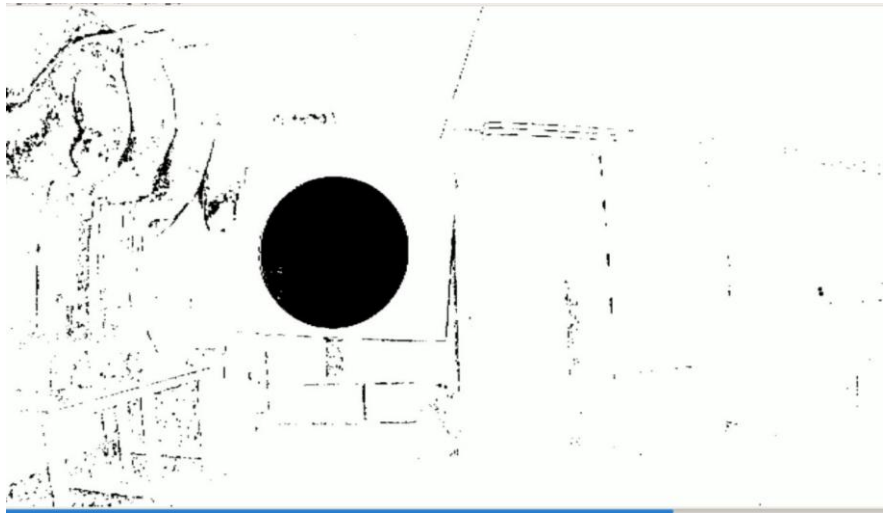
Get maximum value of a given area around a pixel

Using morphological operators in OpenCV

To apply any morphological operator, we will need to define 3 things.

1. *The mask/kernel:* it will define the area of effect of our operation, for example in an erosion the bigger the mask the smaller will the final shape be.
2. *Type of operation:* OpenCV has functions defining the most common operators, so it is a matter of selecting the suitable option
3. *Number of iterations:* It is common to apply these operations recursively, so OpenCV allows setting the number of iterations that are going to be made.

More information can be found here.



Activity 4

Morphological Operations

{Learn, Create, Innovate};

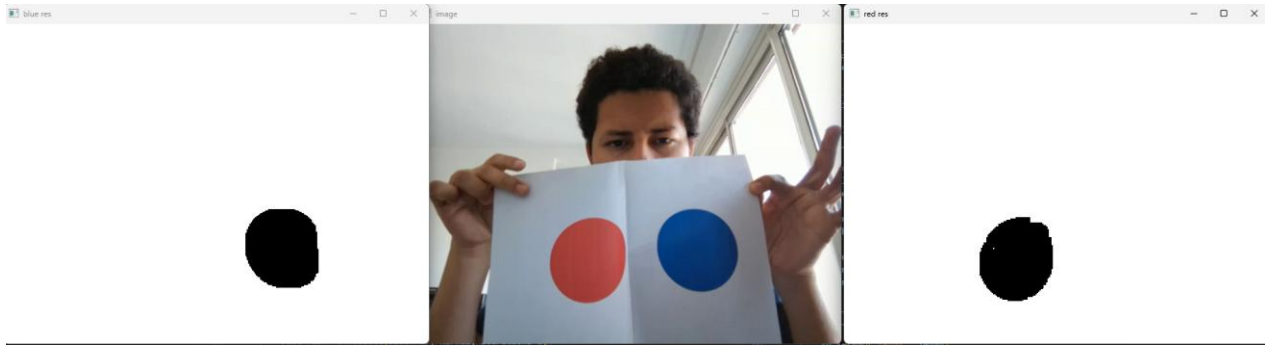
MCR²

Manchester Robotics

Activity 4

- Using the code in Activity 3, morphological operations such as erosion and dilation will be implemented.

1. Transform the masked image to grayscale.
2. Threshold the image to remove areas with low intensity.
3. Erode the image to remove the noise.
4. Dilate the image to smooth edges and enlarge the figures.



Activity 4

```python
import cv2 as cv
import numpy as np

#HSV Values for Red Colour
H_red_low_limit = 0
S_red_low_limit = 95
V_red_low_limit = 130

H_red_up_limit = 5
S_red_up_limit = 255
V_red_up_limit = 255

#HSV Values for Blue Colour
H_blue_low_limit = 95
S_blue_low_limit = 90
V_blue_low_limit = 90

H_blue_up_limit = 110
S_blue_up_limit = 255
V_blue_up_limit = 255

if __name__=="__main__":
    #cap = cv.VideoCapture(2)
    video_path = 'videos/sample_video.mp4'
    cap = cv.VideoCapture(video_path)

    if not cap.isOpened():
        print("Cannot open camera")
```

```python
    #HSV Values for Green Colour
    while True:
        # Reading frame from video or webcam
        ret, image = cap.read()
        # if frame is read correctly ret is True
        if not ret:
            print("Can't receive frame (stream end?). Exiting ...")
            break
        k = cv.waitKey(25) & 0xFF
        if k == 27:
            break

        # Convert BGR to HSV
        hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)

        #Ranges
        red_lower_limit = np.array([H_red_low_limit, S_red_low_limit, V_red_low_limit])
        red_upper_limit = np.array([H_red_up_limit, S_red_up_limit, V_red_up_limit])
        blue_lower_limit = np.array([H_blue_low_limit, S_blue_low_limit, V_blue_low_limit])
        blue_upper_limit = np.array([H_blue_up_limit, S_blue_up_limit, V_blue_up_limit])

        #Masks
        red_mask = cv.inRange(hsv_image, red_lower_limit, red_upper_limit)
        blue_mask = cv.inRange(hsv_image, blue_lower_limit, blue_upper_limit)

        #Partial masks applied to the image (AND Function with the image)
        red_img_result = cv.bitwise_and(image,image,mask=red_mask)
        blue_img_result = cv.bitwise_and(image,image,mask=blue_mask)
```

Activity 4

```
    #Transform to Grayscale
    bw_blue_res = cv.cvtColor(blue_img_result, cv.COLOR_BGR2GRAY)
    bw_red_res = cv.cvtColor(red_img_result, cv.COLOR_BGR2GRAY)

    _ , red_res = cv.threshold(bw_red_res, 30, 255, cv.THRESH_BINARY)
    _ , blue_res = cv.threshold(bw_blue_res, 33, 255, cv.THRESH_BINARY)

    kernel = np.ones((5, 5), np.uint8)
    red_res = cv.erode(red_res, kernel, iterations=3)
    red_res = cv.dilate(red_res, kernel, iterations=3)

    blue_res = cv.erode(blue_res, kernel, iterations=8)
    blue_res = cv.dilate(blue_res, kernel, iterations=8)

    img_res = cv.bitwise_or(red_res, blue_res)

    cv.imshow('image', image)
    cv.imshow('red mask', red_img_result)
    cv.imshow('blue mask', blue_img_result)
    cv.imshow('blue res', blue_res)
    cv.imshow('red res', red_res)
    cv.imshow('result', img_res)

# When everything done, release the captureq
cap.release()
cv.destroyAllWindows()
```

OpenCV

Blob Detection

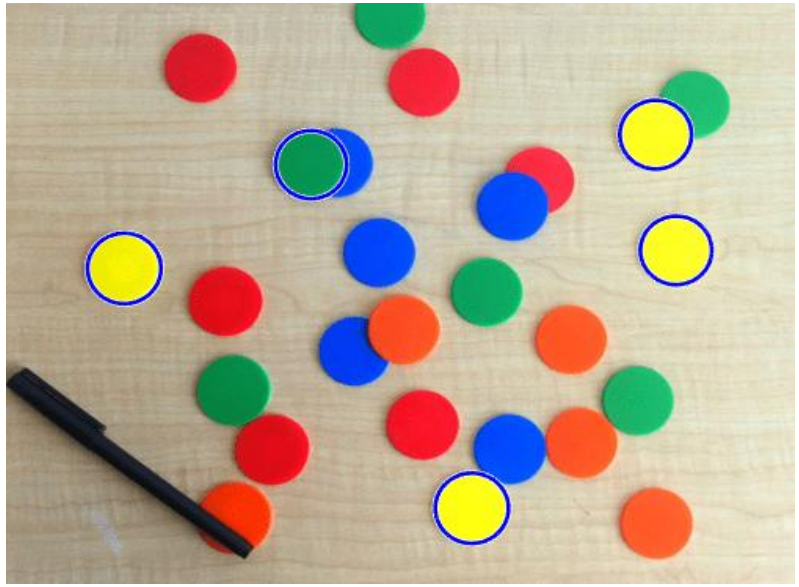
{Learn, Create, Innovate};

MCR²

Manchester Robotics

Blob Detection: What is the problem?

- We aim to detect connected regions in the space with a characteristic shape and colour palette.



Example of the problem

Characteristics to exploit

1. We can look for patches of certain colours
2. We can use size to reject shapes
3. We want to stablish position on the image and area of the objects
4. We can look for different shapes in the image

What are Image Moments?

Image moments are weighted averages of the pixel intensities of an image.

In other words, it is the distribution of matter or intensity around a specific point or axis.

- They provide helpful information about the shape and spatial distribution of objects in images.
- Why are they useful?
- Object centroid (centre of mass)
- Area and orientation
- Shape descriptors (Hu Moments)

General Formula:

$$M_{pq} = \sum_{x} \sum_{y} x^p y^q \, I(x, y)$$

Where:

- p, q = order of the moment
- I = pixel intensity at (x, y)

For the case of a binary image, I are the pixels where the image is non-zero.

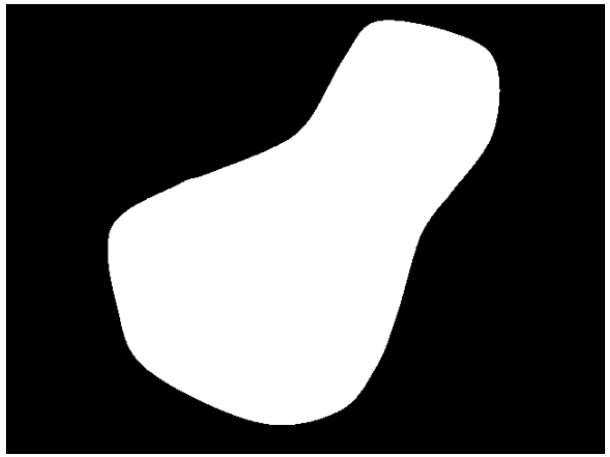
For the case of nonbinary images, it is the average weights/intensity of the pixels. This concept is related to moments of inertia, where heavier objects add more to the moment of inertia about an axis.

What are Image Moments?

Zeroth moment (M_{00})

$$M_{00} = \sum_x \sum_y I(x, y)$$

- Just counts the pixels that are not zero, therefore M_{00} represents the area of the image.

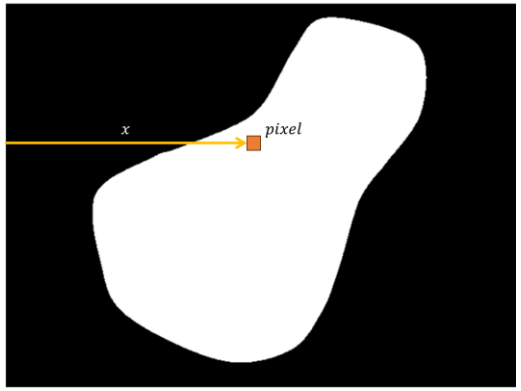


First moment/First moment of Area (M_{10}, M_{01})

$$M_{10} = \sum_x \sum_y x\, I(x, y)$$

$$M_{01} = \sum_x \sum_y y\, I(x, y)$$

- Distance to the area.
- Measures the distribution of area.
- Sum of all the the areas and their respective distances to the centre x or y



What are Image Moments?

Centroids ($\hat{x}$, $\hat{y}$)

$$\hat{x} = \frac{M_{10}}{M_{00}}$$

$$\hat{y} = \frac{M_{01}}{M_{00}}$$

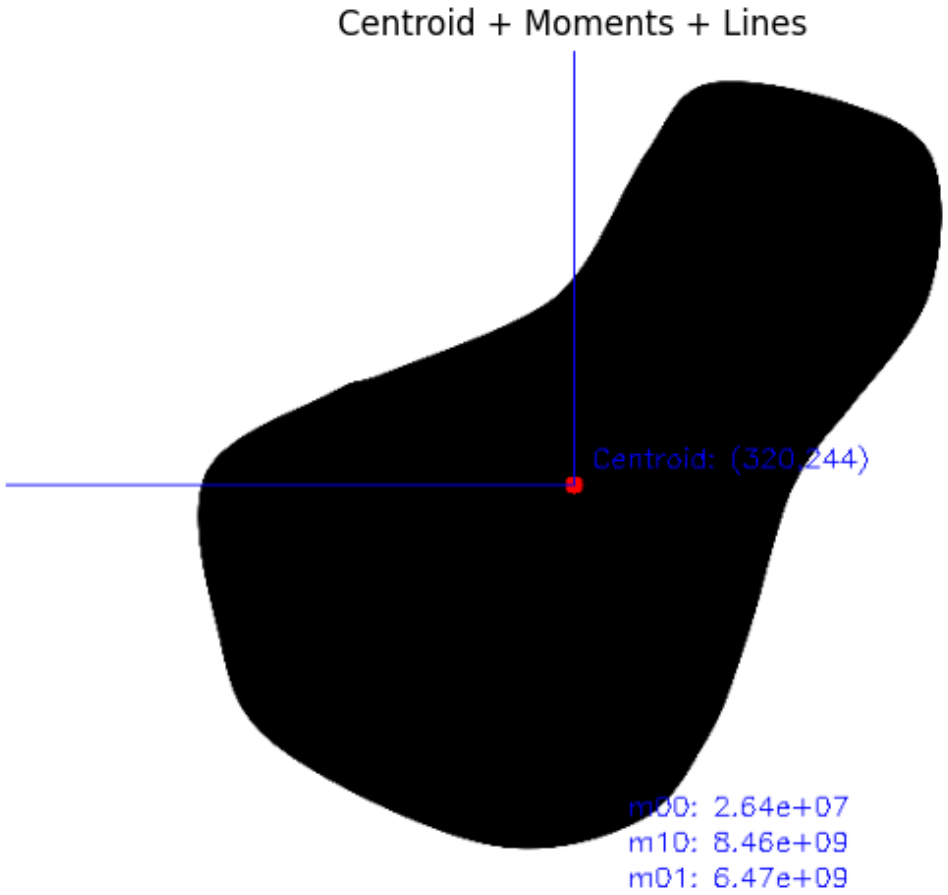
- Using the previous moments, we can define the centroids of the areas ($\hat{x}, \hat{y}$) using the formulas derived earlier.
- This estimate the centre using the average of the distances (sum of all distances divided by the number of distances) of all pixels.

Translated moments (μ_{00})

$$\mu_{pq} = \sum_{x} \sum_{y} (x - \bar{x})^p (y - \bar{y})^q \, I(x, y)$$

- This computes the moments with respect to a given point $\bar{x}$, $\bar{y}$.
- Usually the points centroids $\hat{x}$, $\hat{y}$

What are Image Moments?



Moments

Moments (`cv.moments`):

```
Syntax:
moments = cv2.moments(image, binaryImage=True)
```

- Parameters:
  - image: A binary image (usually from thresholding or edge detection).
  - binaryImage: Boolean flag (default is True). If True, treats all non-zero pixels as 1. If False, uses actual pixel values.

- Returns:
  - A dictionary containing:
    - Spatial moments: m00, m10, m01, m20, m11, m02, m30, m21, m12, m03
    - Central moments: mu20, mu11, mu02, etc.
    - Normalised central moments: nu20, nu11, nu02, etc.

Activity 5

Moments

{Learn, Create, Innovate};

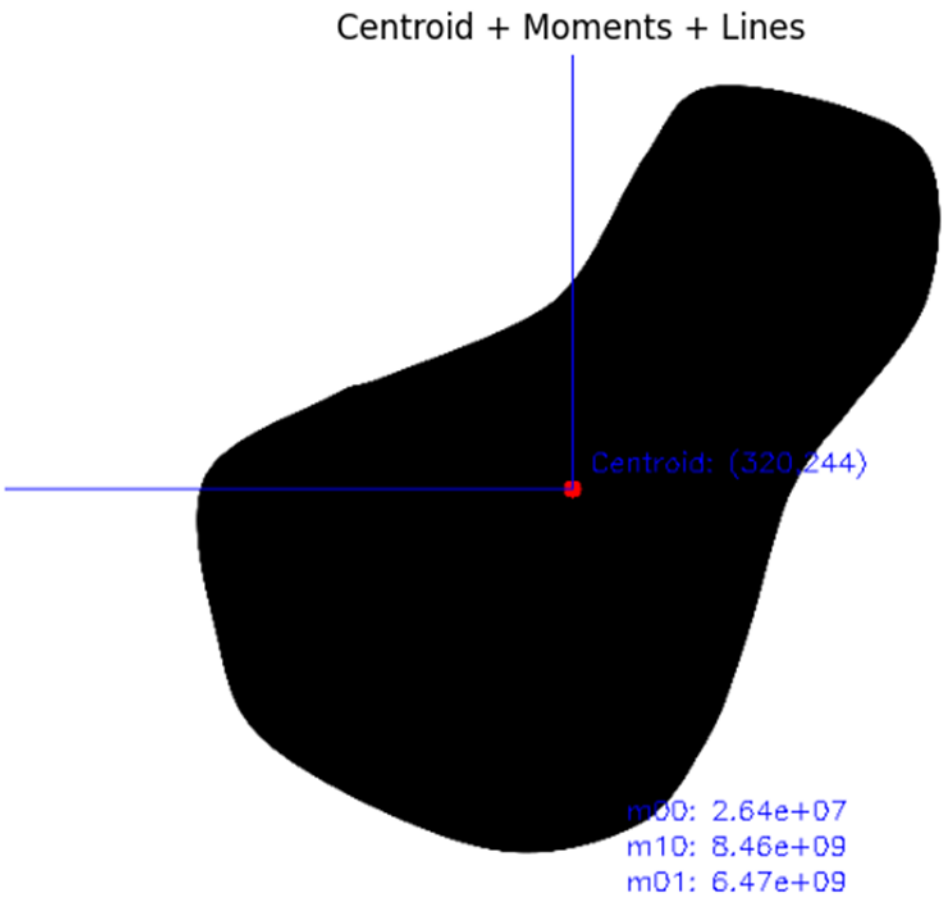
MCR²

Manchester **Robotics**

Activity 5

- For this activity make sure you download *"figure1.png"* into your *images* folder.

1. Transform image to grayscale.
2. Threshold the image to make it binary.
3. Compute moments and centroids.
4. Draw axes and display information.
5. Show the final image



Moments

```python
import cv2
import numpy as np
# ------------------------------------------------------------
# 1. Load image
# ------------------------------------------------------------
# Read the image from disk (BGR format)

img = cv2.imread('images/figure1.png')
assert img is not None, "Error: Image not found!"

# Resize image to a fixed size for consistency
img = cv2.resize(img, (640, 480))

# ------------------------------------------------------------
# 2. Convert to grayscale
# ------------------------------------------------------------
# Moments are usually computed on binary or grayscale images
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ------------------------------------------------------------
# 3. Binary thresholding
# ------------------------------------------------------------
# Convert image to binary:
_, thresh = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV)

# ------------------------------------------------------------
# 4. Compute raw spatial moments
# ------------------------------------------------------------
# Moments describe the spatial distribution of white pixels
moments = cv2.moments(thresh)
```

```python
# ------------------------------------------------------------
# 5. Compute centroid from moments
# ------------------------------------------------------------
# Centroid formulas:
# cx = m10 / m00
# cy = m01 / m00
# m00 corresponds to the object area
if moments["m00"] != 0:
    cx = int(moments["m10"] / moments["m00"])
    cy = int(moments["m01"] / moments["m00"])
else:
    cx, cy = 0, 0  # Safety check (no object detected)

# ------------------------------------------------------------
# 6. Draw centroid and reference axes
# ------------------------------------------------------------
img_with_centroid = img.copy()

# Draw centroid (red dot)
cv2.circle(img_with_centroid, (cx, cy), 5, (0, 0, 255), -1)

# Draw reference lines to visualize centroid position Vertical line (Y-axis reference)
cv2.line(img_with_centroid, (cx, 0), (cx, cy), (255, 0, 0), 1)

# Horizontal line (X-axis reference)
cv2.line(img_with_centroid, (0, cy), (cx, cy), (255, 0, 0), 1)
```

Moments

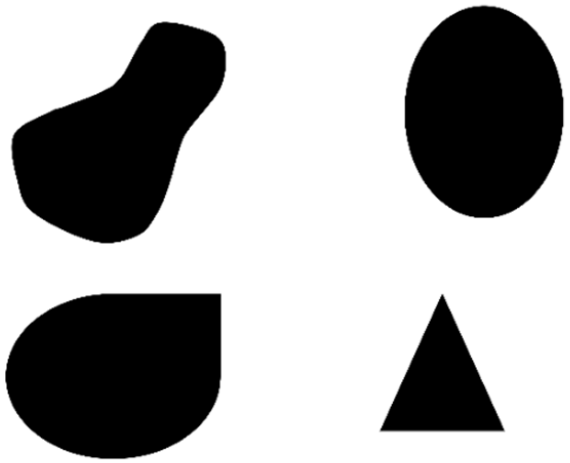
```python
# Label centroid coordinates
cv2.putText(
    img_with_centroid,
    f"Centroid: ({cx}, {cy})",
    (cx + 10, cy - 10),
    cv2.FONT_HERSHEY_SIMPLEX,
    0.5,
    (255, 0, 0),
    1
)

# ---------------------------------------------------------------
# 7. Display moment values on the image
# ---------------------------------------------------------------
info_x = 350
info_y = 430
line_spacing = 20

moment_texts = [
    f"m00 (Area): {moments['m00']:.2e}",
    f"m10: {moments['m10']:.2e}",
    f"m01: {moments['m01']:.2e}"
]
```

```python
for i, text in enumerate(moment_texts):
    cv2.putText(
        img_with_centroid,
        text,
        (info_x, info_y + i * line_spacing),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5,
        (255, 0, 0),
        1
    )
# ---------------------------------------------------------------
# 8. Display results using OpenCV windows
# ---------------------------------------------------------------
cv2.imshow("Original Image + Centroid + Moments", img_with_centroid)
cv2.imshow("Thresholded Image", thresh)

# ---------------------------------------------------------------
# 9. Print moments to terminal (for inspection)
# ---------------------------------------------------------------
print("\n--- Raw Spatial Moments ---")
print(f"m00 (Area): {moments['m00']}")
print(f"m10: {moments['m10']}")
print(f"m01: {moments['m01']}")

# ---------------------------------------------------------------
# 10. Wait and clean up
# ---------------------------------------------------------------
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Connected Component Analysis

- One of the downsides of using Moments is that for images with multiple figures, the algorithm cannot distinguish between them.
- This affects the calculation of moments and the centres of mass (centroids).
- Idea: Split a binary image where pixels are "connected".
- This algorithm is called "Connected-Component Labelling"

Original Image

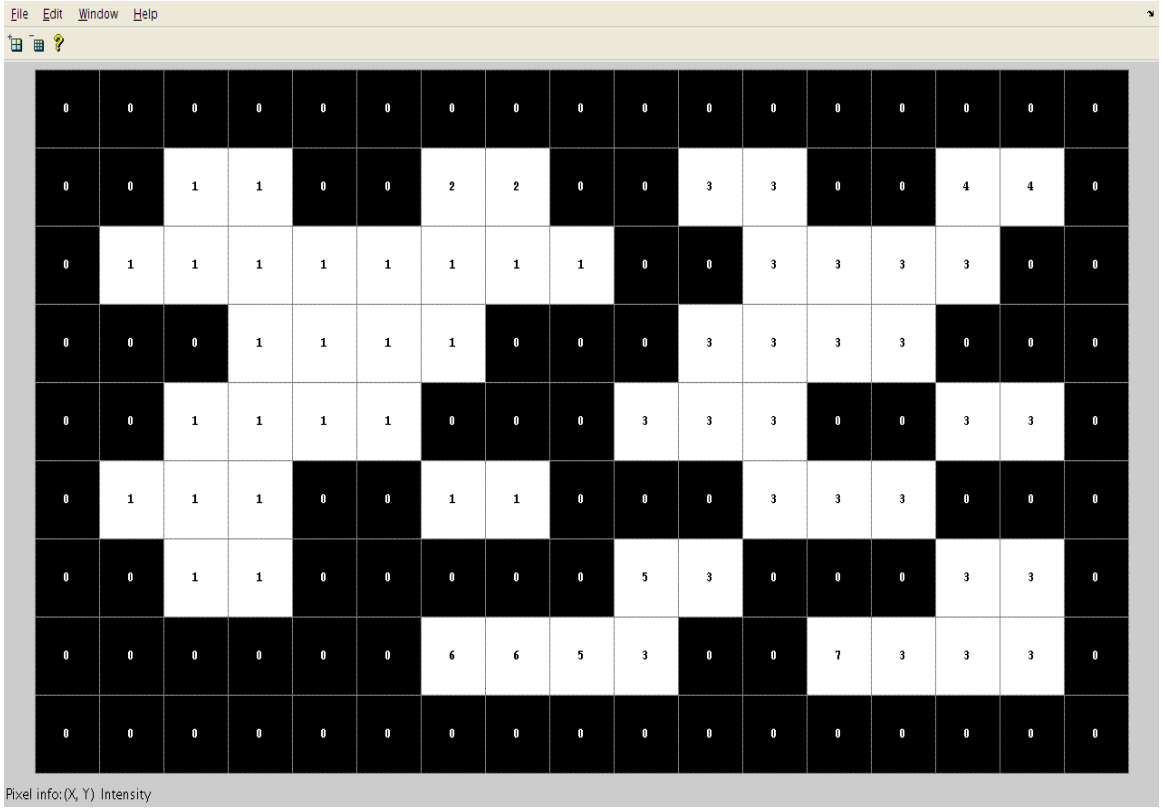


Connected Components Colored

Connected Component Analysis

Connected component labelling

- Connected component labelling is an algorithmic application of graph theory used to determine the connectivity of “blob”-like regions in a binary image.
- The algorithm is used in binary images
- It is useful to detect, separate, and count distinct objects or blobs in an image (e.g., coins, cells, blobs, characters).

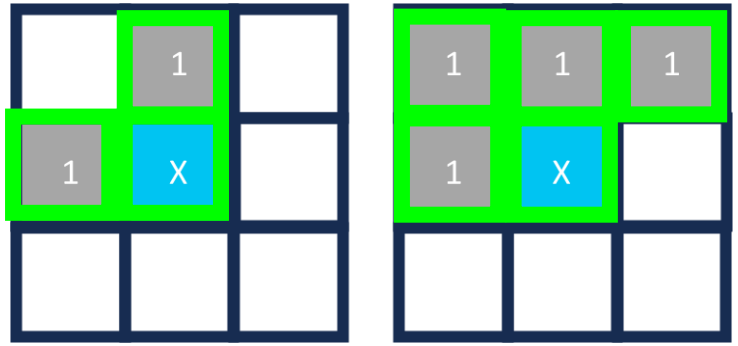


Connected Component Analysis

Connected component labelling

- The algorithm uses a structuring element to verify if pixels "touch/connect" to each other.
- The structuring element scans the image pixel by pixel, from left to right and top to bottom.
- When a foreground pixel is found (pixel==1):
  - The algorithm checks its neighbours (defined by the structuring element)
- If neighbouring foreground pixels exist:
  - They are assigned the same label
- If no neighbours exist:
  - A new label is created
- Labels that turn out to belong to the same object are merged

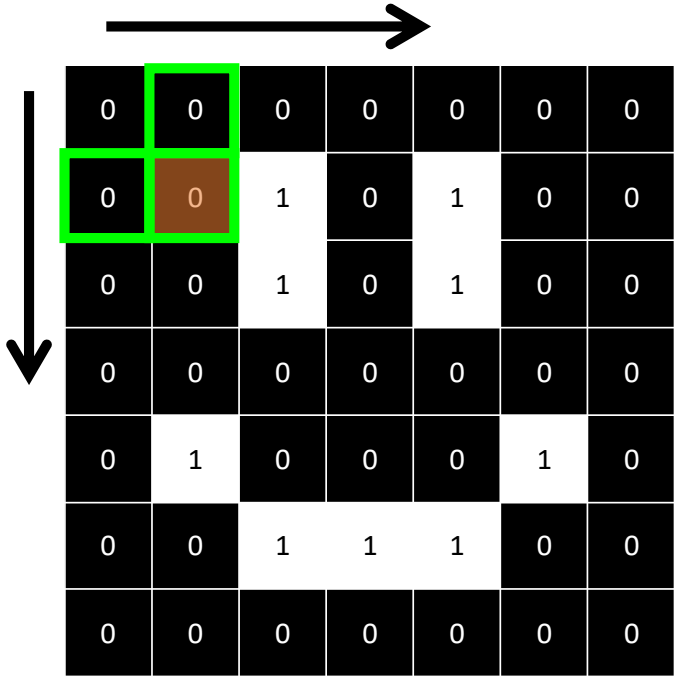
Structuring Element

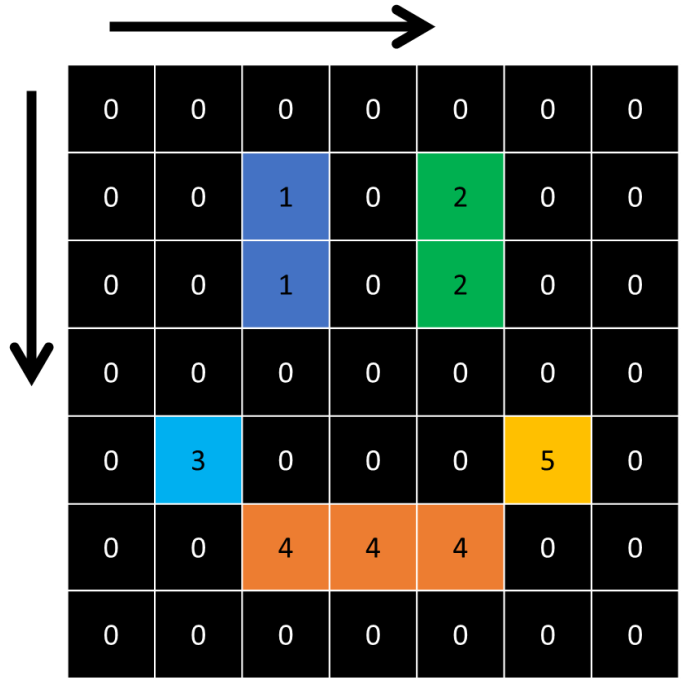


X → current pixel

1 → neighbours that are checked

0 → neighbours not yet visited





Connected Component Analysis

1. Scan the image row by row, from left to right.
2. For each (pixel == 1):
3. Check its neighbouring pixels (up and left neighbours):
    1. Case 1: If no neighbours are labelled → assign a new label.
    2. Case 2: If only one neighbour is labelled → assign that label.
    3. Case 3: If both are labelled and have the same label → Use that label.
    4. Case 4: If both are labelled but have different labels → assign one label and record equivalence.
4. After scanning the full image, resolve equivalences (merge all labels that refer to the same component).
5. Optionally, relabel to ensure labels are sequential (e.g., 1, 2, 3,…).

0	0	0	0	0	0	0
0	0	1	0	2	0	0
0	0	1	0	2	0	0
0	0	0	0	0	0	0
0	x	0	0	0	x	0
0	x	x	x	x	X	0
0	0	0	0	0	0	0

0	0	0	0	0	0	0
0	0	1	0	2	0	0
0	0	1	0	2	0	0
0	0	0	0	0	0	0
0	3	0	0	0	4	0
0	3	x	x	x	X	0
0	0	0	0	0	0	0

0	0	0	0	0	0	0
0	0	1	0	2	0	0
0	0	1	0	2	0	0
0	0	0	0	0	0	0
0	3	0	0	0	4	0
0	3	3	3	3	3	0
0	0	0	0	0	0	0

Equivalence Table
1 -> 1
2 -> 2
3 -> 3
4 -> 4 -> 3

0	0	0	0	0	0	0
0	0	1	0	2	0	0
0	0	1	0	2	0	0
0	0	0	0	0	0	0
0	3	0	0	0	3	0
0	3	3	3	3	3	0
0	0	0	0	0	0	0

Connected Component Analysis

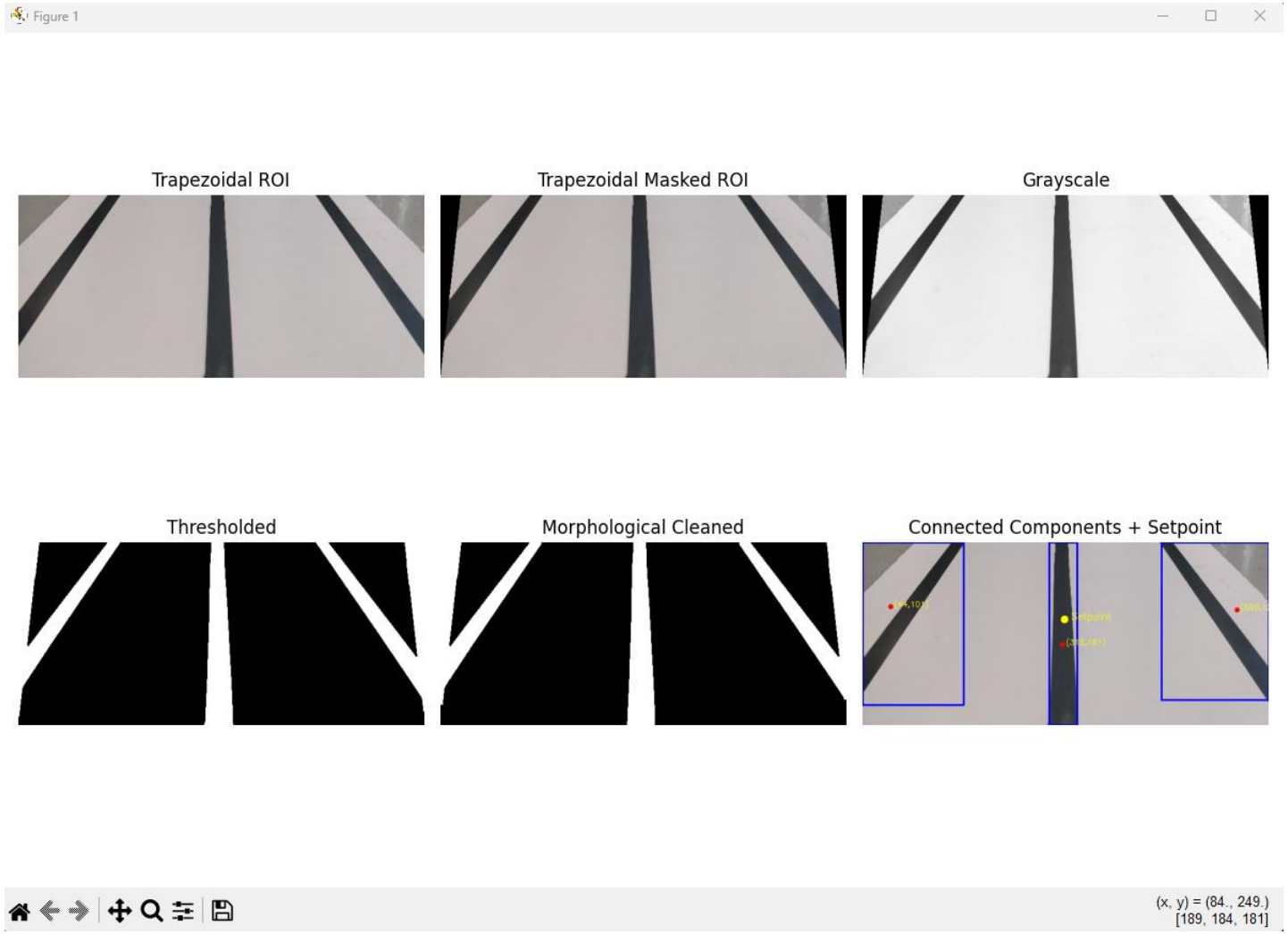
Connected Component Analysis (`cv2.connectedComponentsWithStats`):

```
Syntax:
num_labels, labels, stats, centroids =
cv2.connectedComponentsWithStats(src[, connectivity[,
ltype]])
```

- Parameters:
  - src: Input 8-bit single-channel (binary) image. Non-zero pixels are treated as 1 (foreground), zero pixels as background.
  - connectivity (optional): Type of pixel connectivity:4 → 4-way (top, bottom, left, right)8 → 8-way (includes diagonals)
  - ltype (optional): Output label image type. Default is cv2.CV_32S.
- Returns:
  - num_labels: Number of labels (includes background as label 0).
  - labels: Labeled image with same size as src, where each pixel has a value denoting its component label.
  - stats: Statistics output for each label (including background):
    - stats[i, cv2.CC_STAT_LEFT] – x-coordinate of the bounding box
    - stats[i, cv2.CC_STAT_TOP] – y-coordinate of the bounding box
    - stats[i, cv2.CC_STAT_WIDTH] – width of the bounding box
    - stats[i, cv2.CC_STAT_HEIGHT] – height of the bounding box
    - stats[i, cv2.CC_STAT_AREA] – area (in pixels) of the component
  - centroids: (x, y) coordinates of the centroid of each component.

Examples

- Connected components analysis can be used to detect the edges of a line.
- For this example, the following image was analysed to obtain the centre line.
- This exercise was performed in OpenCV, using the Puzzlebot.



{Learn, Create, Innovate};

Activity 6

Closed Connected Components

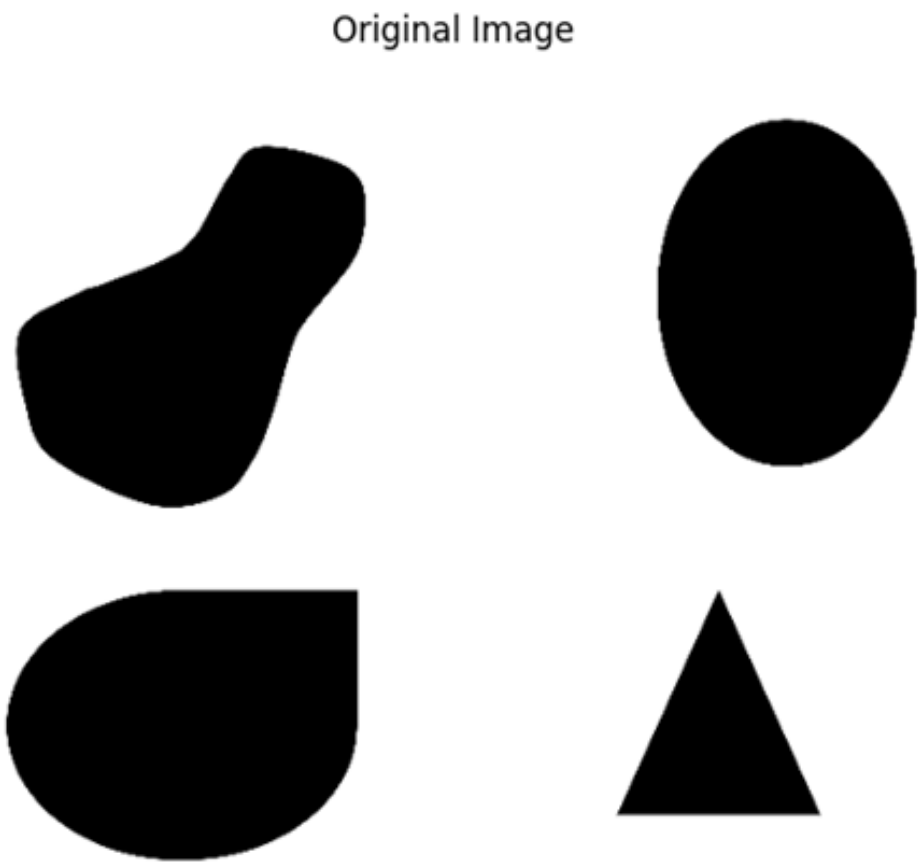
MCR²

Manchester **Robotics**

Activity 5

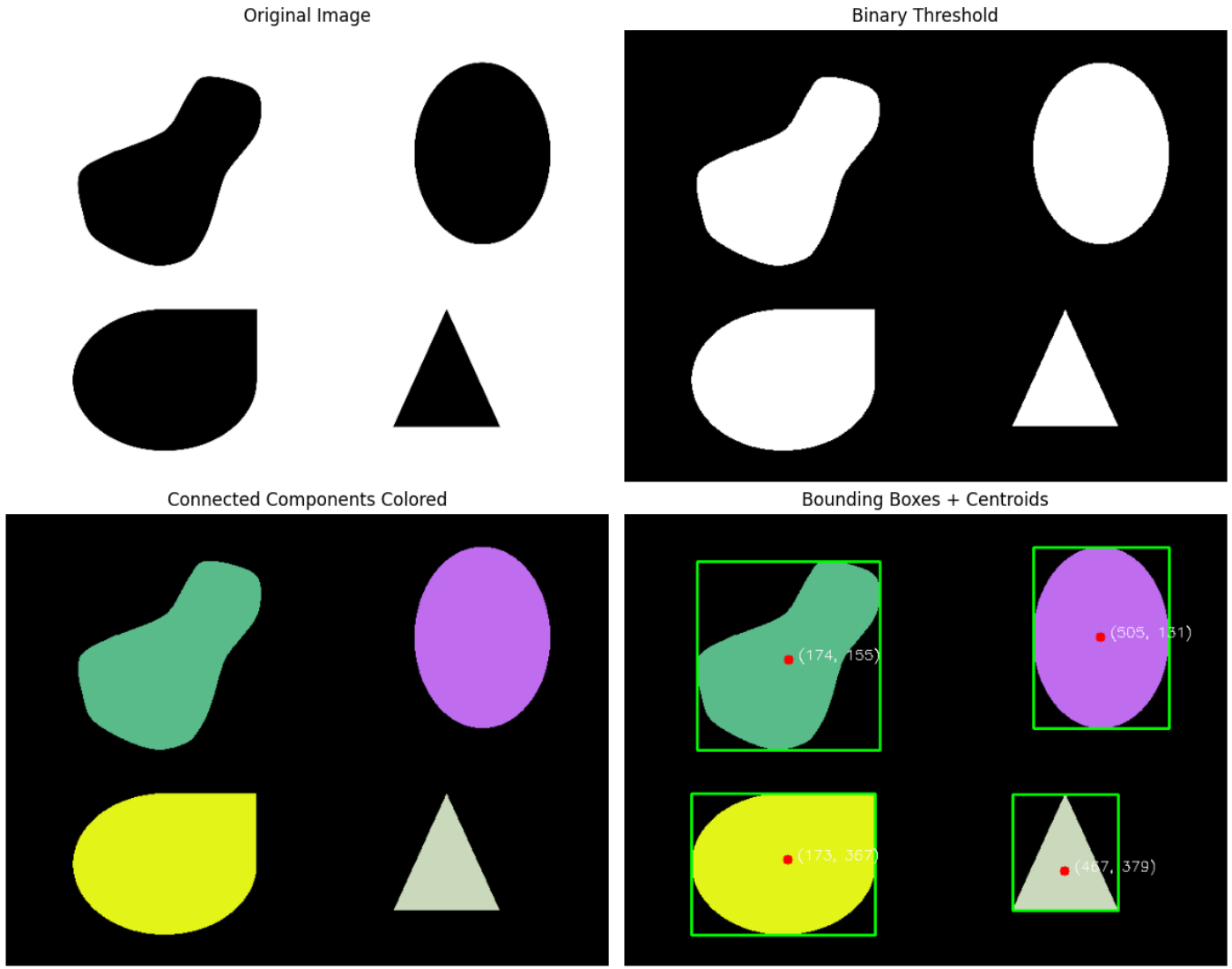
- For this activity make sure you download *"figure2.png"* into your *images* folder.

1. Transform image to grayscale.
2. Threshold the image to make it binary.
3. Compute the closed connected components.
4. Draw centroids, bounding boxes and and display information.
5. Show the final image



```python
import cv2
import numpy as np

# ------------------------------------------------------------
# 1) Load image
# ------------------------------------------------------------

img = cv2.imread("images/figure2.png")
assert img is not None, "Image not found!"
# Resize for consistent display
img = cv2.resize(img, (640, 480))

# ------------------------------------------------------------
# 2) Convert to grayscale
# ------------------------------------------------------------
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

# ------------------------------------------------------------
# 3) Binary thresholding
#    Invert so objects become white (foreground)
# ------------------------------------------------------------
_, binary = cv2.threshold(gray, 1, 255, cv2.THRESH_BINARY_INV)

# ------------------------------------------------------------
# 4) Connected Component Analysis (4-connectivity)
# ------------------------------------------------------------

num_labels, labels, stats, centroids = cv2.connectedComponentsWithStats(
    binary, connectivity=4
)

# ------------------------------------------------------------
# 5) Assign random colors to each component
# ------------------------------------------------------------
colored_components = np.zeros_like(img)

for label_id in range(1, num_labels):  # skip background (label 0)
    mask = (labels == label_id)
    color = np.random.randint(0, 255, size=3)
    colored_components[mask] = color

# ------------------------------------------------------------
# 6) Draw bounding boxes and centroids
# ------------------------------------------------------------
output = colored_components.copy()
for label_id in range(1, num_labels):
    x, y, w, h, area = stats[label_id]
    cx, cy = centroids[label_id]
    # Draw bounding box (green)
    cv2.rectangle(output, (x, y), (x + w, y + h), (0, 255, 0), 2)
    # Draw centroid (red)
    cv2.circle(output, (int(cx), int(cy)), 5, (0, 0, 255), -1)
    # Label centroid coordinates
    cv2.putText(
        output,
        f"({int(cx)}, {int(cy)})",
        (int(cx) + 10, int(cy)),
        cv2.FONT_HERSHEY_SIMPLEX,
        0.5,
        (255, 255, 255),
        1
    )
# ------------------------------------------------------------
# 7) Display all processing stages (OpenCV windows)
# ------------------------------------------------------------
cv2.namedWindow("Original Image", cv2.WINDOW_NORMAL)
cv2.namedWindow("Binary Threshold", cv2.WINDOW_NORMAL)
cv2.namedWindow("Connected Components (Colored)", cv2.WINDOW_NORMAL)
cv2.namedWindow("Bounding Boxes + Centroids", cv2.WINDOW_NORMAL)

cv2.resizeWindow("Original Image", 640, 480)
cv2.resizeWindow("Binary Threshold", 640, 480)
cv2.resizeWindow("Connected Components (Colored)", 640, 480)
cv2.resizeWindow("Bounding Boxes + Centroids", 640, 480)

cv2.imshow("Original Image", img)
cv2.imshow("Binary Threshold", binary)
cv2.imshow("Connected Components (Colored)", colored_components)
cv2.imshow("Bounding Boxes + Centroids", output)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

Connected Component Analysis



{Learn, Create, Innovate};

Activity 7

Vision systems for quality measurement of a Piece

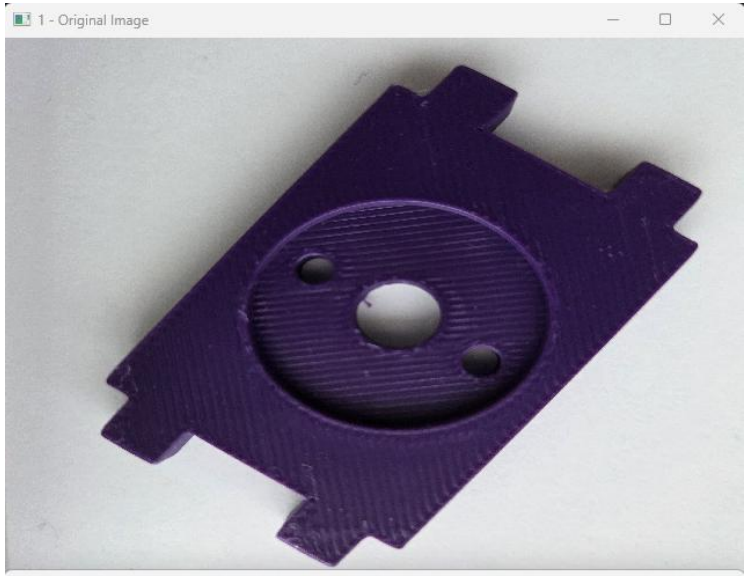
MCR²

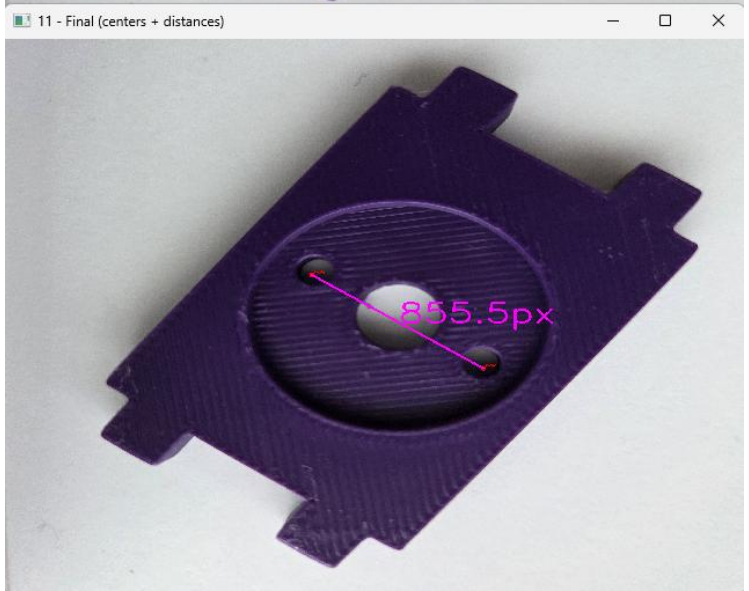
Manchester **Robotics**

Activity 7

Objective

- This activity will demonstrate the use of Moments and connected component analysis to measure the distance between two holes of a real piece to verify its “quality”.
- For this activity, the following image will be analysed to obtain measurements.
- This exercise will be performed in OpenCV.
- *Please note that this is just an example of connected component analysis, measurements might vary due to the position, rotation, and camera/lenses problems.
- This program can be enhanced by knowing the parameters of the camera (camera calibration) and using PCA analysis (principal component analysis) to get the rotation and translation.





Activity 7: Pseudocode

INPUT: image

1) Load image

2) Convert image to HSV

3) Find the “purple” object (piece is “purple” coloured)

   - mask = threshold HSV image for purple colour

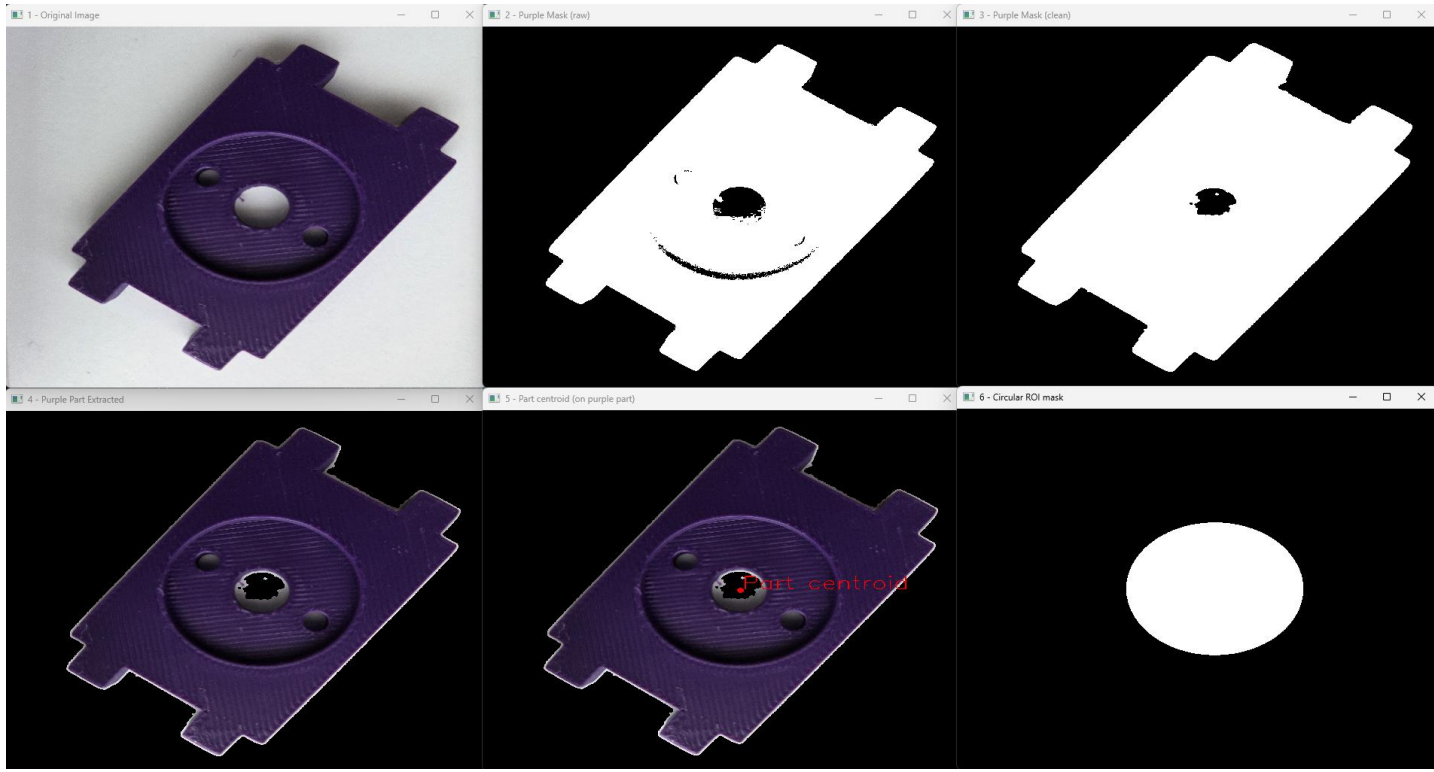
   - clean mask using morphology

4) Find the centre of the “purple” object

   1) centre = centroid of the mask

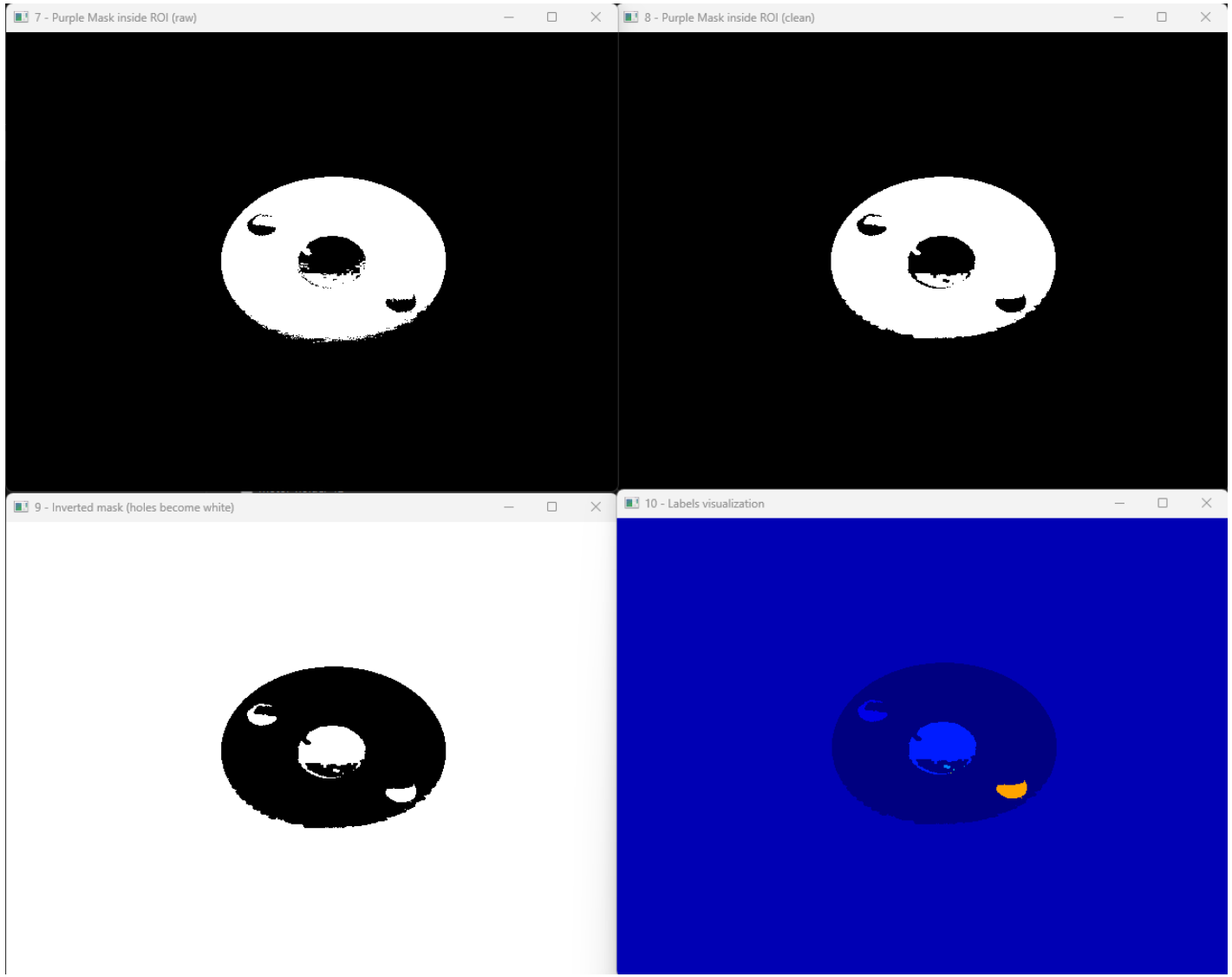
5) Define a circular region of interest (ROI)

   - ROI = circle cantered at the object centre



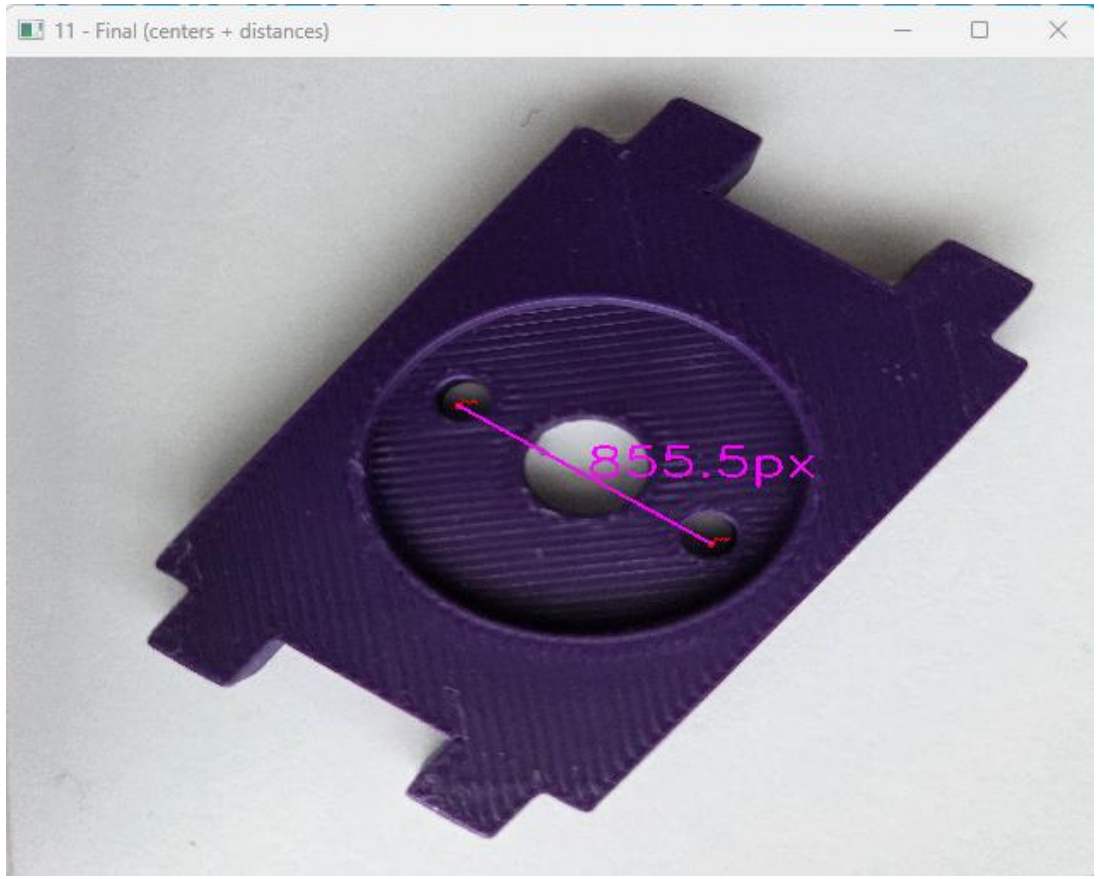
Activity 7: Pseudocode

6) Search for holes only inside the ROI
   - threshold purple colour again (stricter)
   - clean mask with morphology
7) Label connected components
   - invert mask so holes are white
   - label all connected regions
8) Keep only valid holes
   - remove background and small regions
   - store hole centres



Activity 7: Pseudocode

9. Measure distances
    - draw lines between hole centres
    - compute Euclidean distance between them



<u>OUTPUT</u>: image with hole centres and distances

```python
import cv2 as cv
import numpy as np

# ==============================================================
# Helper: display images nicely (resizable windows)
# ==============================================================
def show(win_name, image, w=640, h=480):
    cv.namedWindow(win_name, cv.WINDOW_NORMAL)
    cv.resizeWindow(win_name, w, h)
    cv.imshow(win_name, image)

# ==============================================================
# 0) Parameters (HSV thresholds + morphology + ROI)
# ==============================================================
# --- Purple threshold to find the part (coarse) ---
H1_LOW, S1_LOW, V1_LOW = 120, 15, 15
H1_UP,  S1_UP,  V1_UP  = 160, 255, 255

# --- Purple threshold to refine inside ROI (slightly stricter V) ---
H2_LOW, S2_LOW, V2_LOW = 120, 15, 35
H2_UP,  S2_UP,  V2_UP  = 160, 255, 255

# Morphology settings
KERNEL_SIZE = (5, 5)
DILATE_IT_1 = 10
ERODE_IT_1  = 5

KERNEL_SIZE_2 = (5, 5)
ERODE_IT_2  = 5
DILATE_IT_2 = 5

# Circular ROI radius around the part centroid (pixels)
ROI_RADIUS = 550

# Connected component area filtering (heuristics)
MIN_AREA = 1000
MAX_AREA = 50000
```

```python
# ==============================================================
# 1) Load image
# ==============================================================
img = cv.imread("images/motor_holder/motor_holder_8.jpg")
assert img is not None, "Error: image not found!"

img_draw = img.copy()  # we will draw results on this image
show("1 - Original Image", img)

# ==============================================================
# 2) Convert to HSV (better for color segmentation)
# ==============================================================
hsv = cv.cvtColor(img, cv.COLOR_BGR2HSV)

# ==============================================================
# 3) Step A: Detect the purple part (coarse mask)
# ==============================================================
lower1 = np.array([H1_LOW, S1_LOW, V1_LOW])
upper1 = np.array([H1_UP,  S1_UP,  V1_UP])

purple_mask = cv.inRange(hsv, lower1, upper1)
show("2 - Purple Mask (raw)", purple_mask)

# ==============================================================
# 4) Clean the purple mask (morphology)
#    Dilation fills small gaps, erosion removes noise.
# ==============================================================

kernel = cv.getStructuringElement(cv.MORPH_ELLIPSE, KERNEL_SIZE)
purple_mask_clean = cv.dilate(purple_mask, kernel, iterations=DILATE_IT_1)
purple_mask_clean = cv.erode(purple_mask_clean, kernel,
iterations=ERODE_IT_1)
show("3 - Purple Mask (clean)", purple_mask_clean)

# Visualize masked result (what pixels are selected)
purple_part = cv.bitwise_and(img, img, mask=purple_mask_clean)
show("4 - Purple Part Extracted", purple_part)
```

```python
# ============================================================
# 5) Compute centroid of the purple region using image moments
#    centroid = (m10/m00, m01/m00)
# ============================================================

M = cv.moments(purple_mask_clean)

if M["m00"] != 0:
    part_cx = int(M["m10"] / M["m00"])
    part_cy = int(M["m01"] / M["m00"])
else:
    part_cx, part_cy = 0, 0

# Draw centroid on the extracted part for visualization
purple_part_centroid = purple_part.copy()
cv.circle(purple_part_centroid, (part_cx, part_cy), 20, (0, 0, 255), -1)
cv.putText(purple_part_centroid, "Part centroid", (part_cx + 10, part_cy -
10),
           cv.FONT_HERSHEY_SIMPLEX, 5, (0, 0, 255), 5)
show("5 - Part centroid (on purple part)", purple_part_centroid)

# ============================================================
# 6) Create a circular ROI around the part centroid
#    This limits the search region to avoid detecting unrelated blobs.
# ============================================================

H, W = img.shape[:2]
roi_mask = np.zeros((H, W), dtype=np.uint8)
cv.circle(roi_mask, (part_cx, part_cy), ROI_RADIUS, 255, -1)
show("6 - Circular ROI mask", roi_mask)

# Apply ROI mask to HSV image (we only keep pixels inside the circle)
hsv_roi = cv.bitwise_and(hsv, hsv, mask=roi_mask)
```

```python
# ============================================================
# 7) Step B: Refine purple segmentation inside ROI (stricter)
# ============================================================
lower2 = np.array([H2_LOW, S2_LOW, V2_LOW])
upper2 = np.array([H2_UP,  S2_UP,  V2_UP])

roi_purple_mask = cv.inRange(hsv_roi, lower2, upper2)
show("7 - Purple Mask inside ROI (raw)", roi_purple_mask)

# Clean ROI mask (opening-ish: erosion then dilation)
kernel2 = cv.getStructuringElement(cv.MORPH_ELLIPSE, KERNEL_SIZE_2)
roi_purple_mask_clean = cv.erode(roi_purple_mask, kernel2,
iterations=ERODE_IT_2)
roi_purple_mask_clean = cv.dilate(roi_purple_mask_clean, kernel2,
iterations=DILATE_IT_2)
show("8 - Purple Mask inside ROI (clean)", roi_purple_mask_clean)

# ============================================================
# 8) Connected Components on the INVERTED mask
#    Why invert?
#    - If purple area is white, holes are black.
#    - Inverting makes holes white → easier to label as components.
# ============================================================
inv = cv.bitwise_not(roi_purple_mask_clean)
show("9 - Inverted mask (holes become white)", inv)

num_labels, labels, stats, centroids = cv.connectedComponentsWithStats(inv,
connectivity=8)

# Identify background label by largest area (area column is index 4)
areas = stats[:, 4]
background_label = int(np.argmax(areas))
```

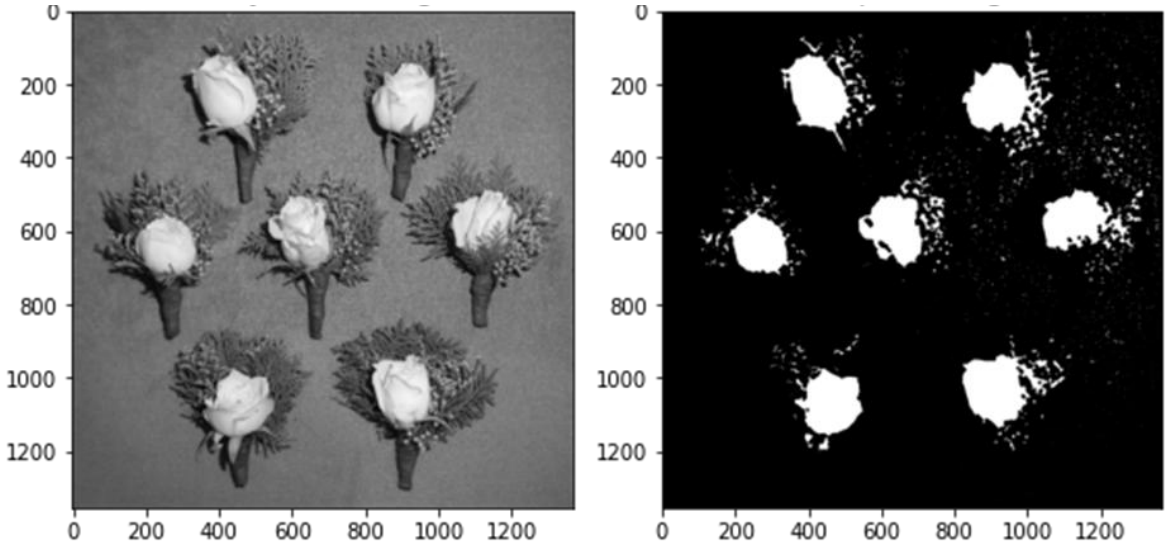
```python
# ============================================================
# 9) Filter components by area and collect centroids
# ============================================================
kept_centers = []

# Visual: color labels (optional, helps students)
labels_vis = cv.applyColorMap((labels * (255 // max(1, num_labels-
1))).astype(np.uint8), cv.COLORMAP_JET)
show("10 - Labels visualization", labels_vis)

for label_id in range(num_labels):

    if label_id == background_label:
        continue

    area = int(stats[label_id, 4])
    if area < MIN_AREA or area > MAX_AREA:
        continue

    cx, cy = centroids[label_id]
    cx_i, cy_i = int(round(cx)), int(round(cy))
    kept_centers.append((cx_i, cy_i))

    # Draw detected hole center on original image
    cv.circle(img_draw, (cx_i, cy_i), 12, (0, 0, 255), -1)
    cv.putText(img_draw, f"hole", (cx_i + 10, cy_i - 10),
               cv.FONT_HERSHEY_SIMPLEX, 0.7, (0, 0, 255), 2)

print("Detected hole centers:", kept_centers)
```

```python
# ------------------------------------------------------------
# 10) Draw distances between every pair of centers
# ------------------------------------------------------------
n = len(kept_centers)

for i in range(n):
    x1, y1 = kept_centers[i]

    for j in range(i + 1, n):
        x2, y2 = kept_centers[j]

        # Draw line between centers
        cv.line(img_draw, (x1, y1), (x2, y2), (255, 0, 255), 10)

        # Euclidean distance in pixels
        d = float(np.hypot(x2 - x1, y2 - y1))

        # Put text at midpoint
        mx, my = (x1 + x2) // 2, (y1 + y2) // 2
        cv.putText(img_draw, f"{d:.1f}px", (mx + 5, my + 5),
                   cv.FONT_HERSHEY_SIMPLEX, 5, (255, 0, 255), 10)

# ============================================================
# 11) Final visualization
# ============================================================
show("11 - Final (centers + distances)", img_draw)
cv.waitKey(0)
cv.destroyAllWindows()
```

What is blob?

- A Blob is a group of connected pixels in an image that share some common property.
- Usually are areas in an image that are visually distinct, often brighter/darker.
- In the image, the dark connected regions are blobs, and the goal of blob detection is to identify and mark these regions.
- Useful to detect objects (balls, screws, parts).
- Tracking moving elements.
- Obstacle detection.

➡️ Blob = “Binary Large Object”

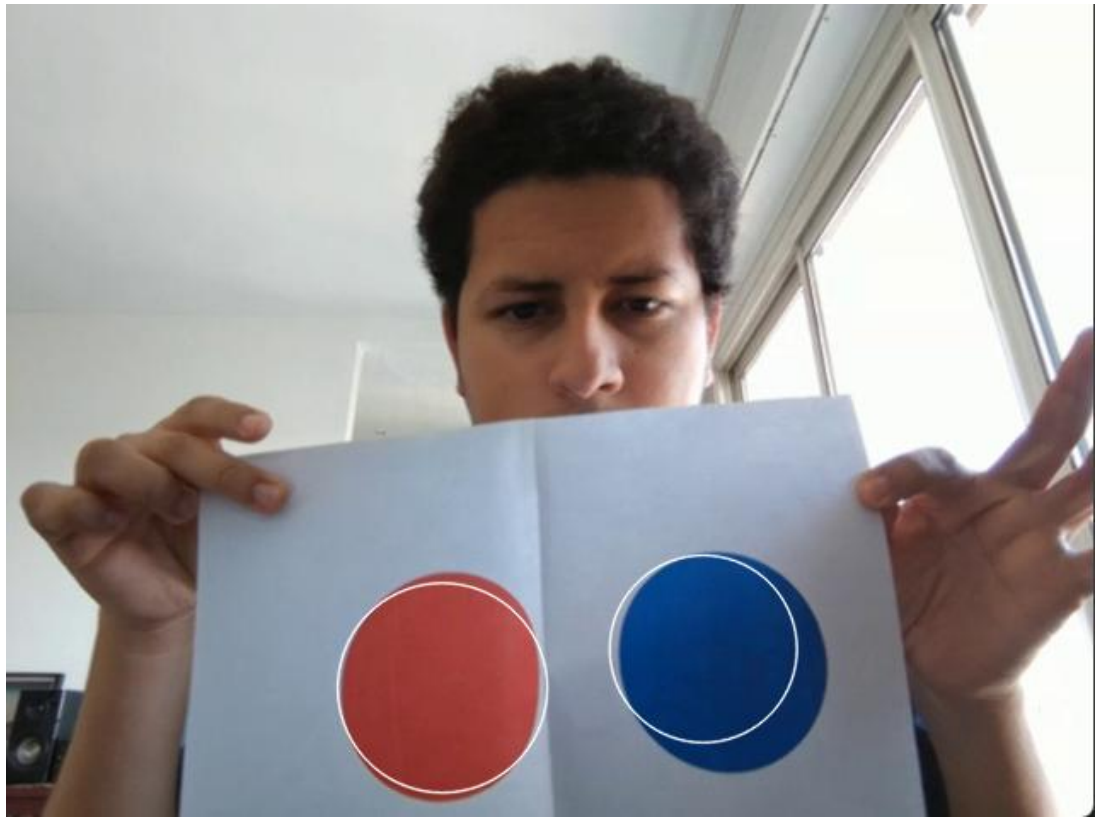


Grayscale image

Resulting blobs after processing

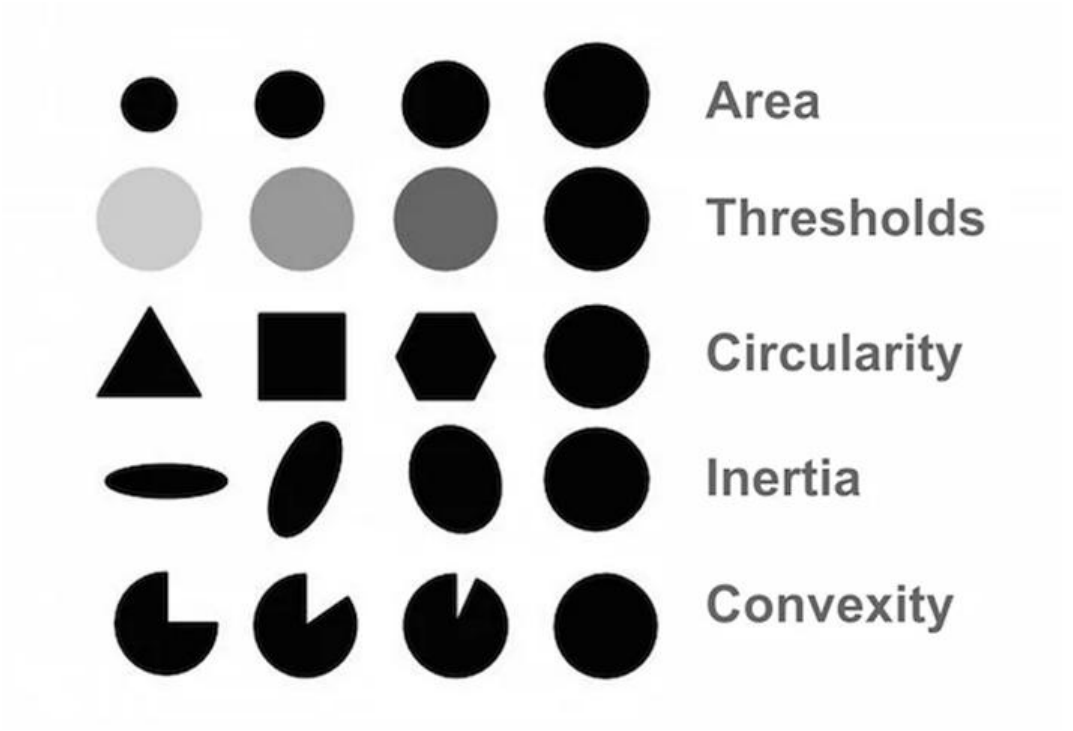
What is blob?

- Blob detection involves five steps:
  - **Thresholding**: convert the input image to a binary image through thresholding.
  - **Grouping**: Find and cluster connected pixels (regions), by making use of the "connected component" function. The centre is identified for each of the extracted groups of pixels and the value for the centres is computed.
  - **Merging**: Merges all nearby clusters.
  - **Radius and centres**: Compute radiuses and centres of the blobs.
  - **Filtering**: Keep blobs matching specific criteria:
    - Size
    - Circularity
    - Convexity
    - Inertia ratio (elongation), etc.



Blob Filtering

- **Color [0,255]:** Compares the intensity of a binary image at the center of a blob to the parameter *blobColor*.
    - blobColor [0,255] to extract dark blobs and light blobs respectively.
- **Area [0, maxArea]:** Extracted blobs have an area between *minArea* and *maxArea* parameters.
- **Circularity [0,1]:** Extracted blobs have circularity between *minCircularity* and *maxCircularity (Circularity of the Circle is 1).*
- **Ratio [0,1]:** Measures the inertia (elongation)of the blob. Extracted blobs have this ratio between *minInertiaRatio* and *maxInertiaRatio.*
- **Convexity [0,1]:** Extracted blobs have convexity (area / area of blob convex hull) between *minConvexity* and *maxConvexity.*



Tuning Parameters

Parameter	What it Does	Typical Use
Threshold	Minimum grayscale intensity for detection.	Lower limit
filterByArea	Only blobs within specific area.	Filter noise
filterByCircularity	Select blobs close to circular shape.	Detect more specific shapes such as circles
filterByConvexity	How convex the blob shape is.	Avoid hollow shapes
filterByInertia	Measures blob elongation.	Exclude thin blobs
filterByColor	Check colour of the blob	Exclude blobs of different colours.

Activity 8

*I Y` \U7\R: UdUSdY^*

{Learn, Create, Innovate};

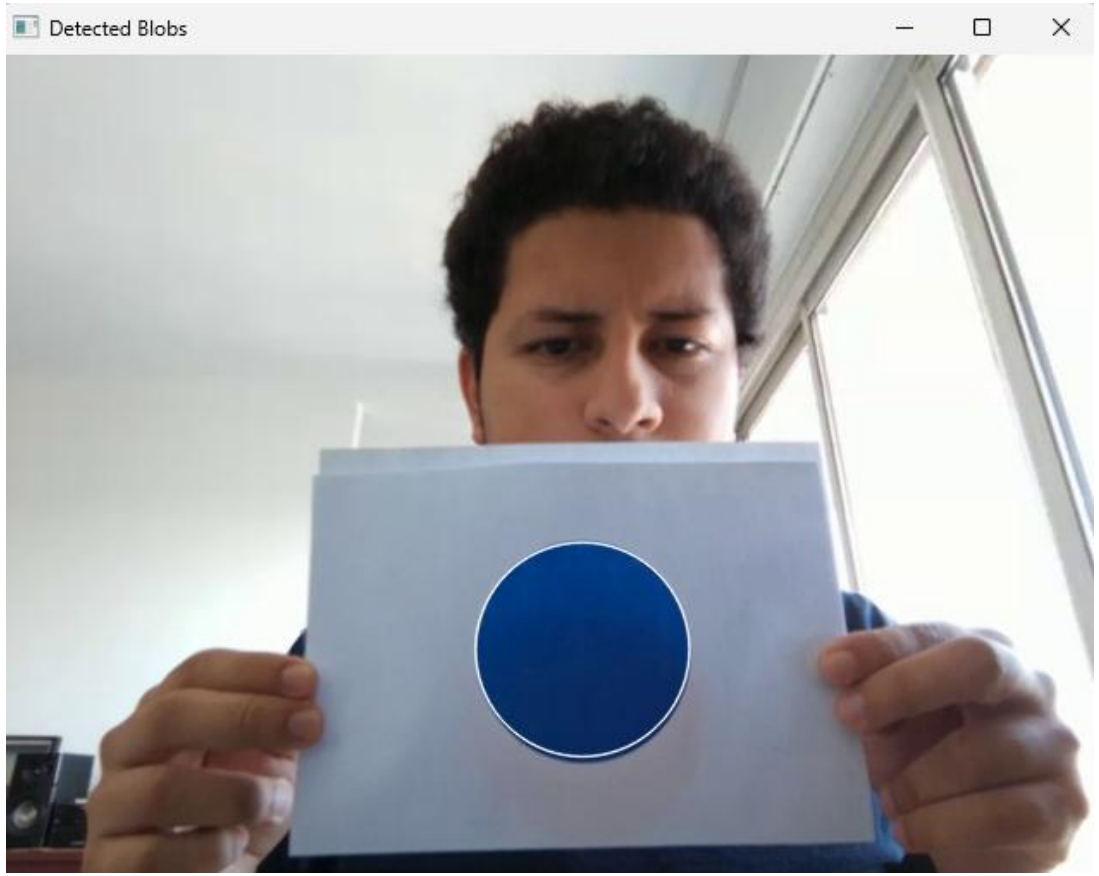
Manchester Robotics

Activity 8

Using the code in Activity 4, blob detection will be used to detect the blue blob in a video.

For this activity "`sample_video.mp4`" must be inside the folder "videos"

1. Transform the masked image to grayscale.
2. Threshold the image to remove areas with low intensity.
3. Erode the image to remove the noise.
4. Dilate the image to smooth edges and enlarge the figures.
5. Detect blobs
6. Print centres and blobs.



```python
import cv2 as cv
import numpy as np

# HSV Ranges for detecting BLUE color
H_blue_low_limit = 95
S_blue_low_limit = 90
V_blue_low_limit = 90
H_blue_up_limit = 110
S_blue_up_limit = 255
V_blue_up_limit = 255

# Configuration of OpenCV SimpleBlobDetector Parameters
blob_detec_params = cv.SimpleBlobDetector_Params()

# Thresholds for intensity difference (pixel brightness) during detection
blob_detec_params.minThreshold = 30
blob_detec_params.maxThreshold = 255

# Filter blobs by color (white blobs if 255)
blob_detec_params.filterByColor = True
blob_detec_params.blobColor = 255

# Filter blobs by area (size in pixels)
blob_detec_params.filterByArea = True
blob_detec_params.minArea = 30         # Minimum blob area
blob_detec_params.maxArea = 10000000   # Maximum blob area

# Filter blobs by convexity (how convex the shape is)
blob_detec_params.filterByConvexity = True
blob_detec_params.minConvexity = 0.1
blob_detec_params.maxConvexity = 1

# Filter blobs by circularity (how circular the blob is)
blob_detec_params.filterByCircularity = True
blob_detec_params.minCircularity = 0.5
blob_detec_params.maxCircularity = 1

# Filter blobs by inertia (how elongated the blob is)
blob_detec_params.filterByInertia = True
blob_detec_params.minInertiaRatio = 0.5
blob_detec_params.maxInertiaRatio = 1
```

```python
# Main Code Execution Starts Here
if __name__ == "__main__":

    # Start video capture
    # You can switch to a webcam by using cv.VideoCapture(0)
    video_path = 'videos/sample_video.mp4'
    cap = cv.VideoCapture(video_path)

    if not cap.isOpened():
        print("Cannot open camera or video file")

    # Create the blob detector object with configured parameters
    detector = cv.SimpleBlobDetector_create(blob_detec_params)

    # Main loop to process each frame
    while True:

        # Capture frame from video
        ret, image = cap.read()

        # Exit the loop if no frames are returned (end of video)
        if not ret:
            print("Can't receive frame (stream end?). Exiting ...")
            break

        # Check for ESC key press to exit early
        k = cv.waitKey(25) & 0xFF
        if k == 27:
            break
```

```python
# Step 1: Convert image from BGR to HSV color space
hsv_image = cv.cvtColor(image, cv.COLOR_BGR2HSV)

        # Step 2: Create masks for red and blue colors
# Blue mask: a single range
blue_lower_limit = np.array([H_blue_low_limit,
S_blue_low_limit, V_blue_low_limit])
blue_upper_limit = np.array([H_blue_up_limit, S_blue_up_limit,
V_blue_up_limit])

        # Apply color thresholds to create binary masks
blue_mask = cv.inRange(hsv_image, blue_lower_limit,
blue_upper_limit)

        # Step 3: Apply masks to the original image for
visualization
# Apply blue mask
blue_img_result = cv.bitwise_and(image, image, mask=blue_mask)

        # Step 4: Convert the masked images to grayscale for
further processing
bw_blue_res = cv.cvtColor(blue_img_result, cv.COLOR_BGR2GRAY)

        # Step 5: Apply binary thresholding to highlight blobs
in binary images
_, blue_res = cv.threshold(bw_blue_res, 33, 255,
cv.THRESH_BINARY)
```

```python
        # Step 6: Perform morphological operations to clean up noise
        # Create a kernel for morphological operations
        kernel = np.ones((5, 5), np.uint8)

                # Clean blue mask: more aggressive clean-up
        blue_res = cv.erode(blue_res, kernel, iterations=8)
        blue_res = cv.dilate(blue_res, kernel, iterations=8)

                # Step 7: Detect blobs in the cleaned binary images
        bluekeypoints = detector.detect(blue_res)

                # Step 9: Draw keypoints (blobs) on the original image
        view_blobs = cv.drawKeypoints(image, bluekeypoints,
       np.array([]),
                                      (255, 255, 255),
       cv.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)

                # Step 10: Display the images for
       visualization/debugging
        #cv.imshow('Original Image', image)
        #cv.imshow('Blue Mask Applied', blue_img_result)
        #cv.imshow('Blue Binary Mask', blue_res)
        cv.imshow('Detected Blobs', view_blobs)

            # Step 11: Release resources and close windows when done
    cap.release()
    cv.destroyAllWindows()
```

JXQ^[i _e

{Learn, Create, Innovate};

MCR²

Manchester Robotics

T&C

Terms and conditions

{Learn, Create, Innovate};

MCR²

Manchester Robotics

Terms and conditions

- *THE PIECES, IMAGES, VIDEOS, DOCUMENTATION, ETC. SHOWN HERE ARE FOR INFORMATIVE PURPOSES ONLY. THE DESIGN IS PROPRIETARY AND CONFIDENTIAL TO MANCHESTER ROBOTICS LTD. (MCR2). THE INFORMATION, CODE, SIMULATORS, DRAWINGS, VIDEOS PRESENTATIONS ETC. CONTAINED IN THIS PRESENTATION IS THE SOLE PROPERTY OF MANCHESTER ROBOTICS LTD. ANY REPRODUCTION, RESELL, REDISTRIBUTION OR USAGE IN PART OR AS A WHOLE WITHOUT THE WRITTEN PERMISSION OF MANCHESTER ROBOTICS LTD. IS STRICTLY PROHIBITED.*
- *THIS PRESENTATION MAY CONTAIN LINKS TO OTHER WEBSITES OR CONTENT BELONGING TO OR ORIGINATING FROM THIRD PARTIES OR LINKS TO WEBSITES AND FEATURES IN BANNERS OR OTHER ADVERTISING. SUCH EXTERNAL LINKS ARE NOT INVESTIGATED, MONITORED, OR CHECKED FOR ACCURACY, ADEQUACY, VALIDITY, RELIABILITY, AVAILABILITY OR COMPLETENESS BY US.*
- *WE DO NOT WARRANT, ENDORSE, GUARANTEE, OR ASSUME RESPONSIBILITY FOR THE ACCURACY OR RELIABILITY OF ANY INFORMATION OFFERED BY THIRD-PARTY WEBSITES LINKED THROUGH THE SITE OR ANY WEBSITE OR FEATURE LINKED IN ANY BANNER OR OTHER ADVERTISING.*